UNIVERSITY RESEARCH PAPER · OPERATING SYSTEMS

Completely Fair Scheduler (CFS): Design, Implementation, Performance, and Comparison with Real-Time Scheduling Algorithms

A Comprehensive Study of CPU Scheduling in Modern Operating Systems

STUDENT FULL NAME
Mohammadreza Azodi

INSTRUCTOR / PROFESSOR NAME
Majid Shamshirgaran

UNIVERSITY NAME
Eshragh University

DEPARTMENT / FACULTY
Computer

COURSE NAME
Embedding Systems

ACADEMIC SEMESTER
ONE

SUBMISSION DATE
2026 OCT 22

Document type: Complete academic research paper · **Format:** A4, IEEE-style citations · **Version:** 1.0

Abstract

CPU scheduling is a central function of every general-purpose operating system. It determines which runnable task receives processor time, for how long, and on which core, and therefore directly influences throughput, latency, fairness, and predictability. This paper provides a comprehensive study of the **Completely Fair Scheduler (CFS)** , the fair scheduling class that Linux used as its default desktop and server scheduler from kernel 2.6.23 until the gradual introduction of the **Earliest Eligible Virtual Deadline First (EEVDF)** algorithm in Linux 6.6. The paper explains the conceptual model of CFS—an "ideal, precise multi-tasking CPU" approximated through virtual runtime (`vruntime`), task weights derived from nice values, and a time-ordered red-black tree—and then examines the practical workflow, pseudocode, multiprocessor load balancing, group scheduling, and the limitations of the design. A separate chapter compares CFS with the classical real-time algorithms **Rate Monotonic (RM)** and **Earliest Deadline First (EDF)** , including their schedulability bounds and assumptions. The paper emphasizes that CFS is a fairness-oriented general-purpose scheduler and does not provide hard real-time deadline guarantees. It also distinguishes classic CFS from the newer EEVDF-based fair scheduling class and explains why kernel-version awareness is essential when interpreting Linux scheduling behavior. Illustrative examples, SVG diagrams, tables, and a reproducible Linux demonstration are included. All quantitative examples are explicitly identified as educational illustrations rather than benchmark measurements.

Keywords

CPU scheduling · Completely Fair Scheduler · CFS · EEVDF · Linux kernel · virtual runtime · red-black tree · proportional fairness · Rate Monotonic · Earliest Deadline First · real-time scheduling · multicore · load balancing · cgroups

Table of Contents

Abstract i

Keywords i

List of Figures ii

List of Tables iii

List of Abbreviations iv

1. Introduction 1

    1.1 The Role of CPU Scheduling 1

    1.2 Why Scheduling Matters in Multitasking Environments 2

    1.3 Fairness, Throughput, Responsiveness, and Predictability 2

    1.4 Historical Evolution of Linux CPU Scheduling 3

    1.5 Motivation Behind CFS 3

    1.6 Research Objectives, Scope, and Methodology 4

    1.7 Organization of the Paper 4

2. Fundamentals of CPU Scheduling 5

    2.1 Processes, Threads, Tasks, and CPU Bursts 5

    2.2 Preemptive versus Non-Preemptive Scheduling 5

    2.3 Scheduling Queues and Context Switching 6

    2.4 Performance Metrics and Worked Examples 6

    2.5 Interactive versus CPU-Intensive Workloads 8

    2.6 General-Purpose versus Real-Time Scheduling 8

    2.7 SMP and Multicore Scheduling Fundamentals 9

3. The Completely Fair Scheduler (CFS) 10

    3.1 Ideal Multitasking 10

    3.2 Proportional Fairness 10

    3.3 Virtual Runtime (`vruntime`) 11

    3.4 Task Weights and Scheduling Priority 11

    3.5 Nice Values and CPU Allocation 12

    3.6 Scheduling Entities 12

    3.7 The Runnable-Task Structure and Red-Black Tree 13

    3.8 Choosing the Next Task 14

    3.9 Updating Virtual Runtime 14

    3.10 Preemption and Scheduling Decisions 15

    3.11 Task Creation, Termination, Blocking, and Waking 15

    3.12 Load Balancing and CPU Affinity 16

    3.13 Group Scheduling and Hierarchical Fairness 16

    3.14 CFS and Linux Scheduling Classes 17

3.15 Classic CFS versus EEVDF 17
4. CFS Algorithm Workflow 19
4.1 Step-by-Step Workflow 19
4.2 Required Flowchart 20
4.3 Conceptual Model versus Implementation 21
5. CFS Pseudocode and Worked Example 22
5.1 Conceptual Pseudocode 22
5.2 Worked Example with Three Tasks 23
5.3 Expected versus Observed Allocation 25
6. Main Features and Advantages of CFS 27
7. Multiprocessing and Multicore Scheduling 30
8. Performance Evaluation and Graphs 33
9. Comparison of CFS, RM, and EDF 37
10. Limitations and Disadvantages of CFS 41
11. Starvation, Deadlock, and Priority Inversion 43
12. Security, Reliability, and Practical Considerations 46
13. Practical Linux Demonstration 48
14. Discussion and Future Directions 50
15. Conclusion 52
References 53

List of Figures

Figure 3.1 — Conceptual relationship between actual runtime, weight, and virtual runtime p. 12

Figure 3.2 — Simplified red-black tree of runnable scheduling entities keyed by `vruntime` p. 13

Figure 4.1 — CFS scheduling workflow flowchart p. 20

Figure 5.1 — Gantt chart for the worked three-task example p. 25

Figure 7.1 — Per-CPU run queues and task migration in an SMP system p. 31

Figure 8.1 — Illustrative CPU allocation among tasks with different nice values p. 34

Figure 8.2 — Illustrative evolution of virtual runtime for three tasks p. 34

Figure 8.3 — Gantt chart of illustrative CFS execution intervals p. 35

Figure 8.4 — Qualitative comparison of CFS, RM, and EDF scheduling characteristics p. 35

Figure 8.5 — Load balancing across four CPU cores p. 36

List of Tables

Table 2.1 — Comparison of preemptive and non-preemptive scheduling p. 5

Table 2.2 — Scheduling metrics and their definitions p. 7

Table 3.1 — Major CFS concepts and terminology p. 11

Table 3.2 — Illustrative nice values, weights, and relative CPU shares p. 12

Table 5.1 — Initial task parameters for the worked example p. 23

Table 5.2 — Virtual runtime evolution in the worked example p. 24

Table 6.1 — Advantages and disadvantages of CFS p. 28

Table 9.1 — Detailed comparison of CFS, RM, and EDF p. 38

Table 11.1 — Starvation versus deadlock versus priority inversion p. 45

Table 13.1 — Linux scheduling tools and their purposes p. 48

Table 3.3 — Classic CFS versus newer EEVDF-related concepts p. 18

Table 15.1 — Summary of the main findings p. 52

List of Abbreviations

ABBREVIATION	MEANING
CFS	Completely Fair Scheduler
EEVDF	Earliest Eligible Virtual Deadline First
RM	Rate Monotonic scheduling
EDF	Earliest Deadline First scheduling
SMP	Symmetric Multiprocessing
NUMA	Non-Uniform Memory Access
vruntime	Virtual runtime
VRT	Virtual run time (EEVDF lag decay)
VD	Virtual deadline (EEVDF)
RT	Real-time
SCHED_NORMAL	Linux default fair scheduling policy
SCHED_FIFO	POSIX real-time FIFO policy
SCHED_RR	POSIX real-time round-robin policy
SCHED_DEADLINE	Linux deadline scheduling policy
cgroup	Linux control group
PELT	Per-Entity Load Tracking

1. Introduction

1.1 The Role of CPU Scheduling

An operating system is responsible for multiplexing a finite set of physical processors among a potentially large and dynamically changing set of runnable tasks. The component that makes the moment-to-moment decision of *which task runs next* is the **CPU scheduler**. Scheduling is not a peripheral concern: it sits at the intersection of performance, fairness, responsiveness, energy consumption, and isolation. A scheduler that consistently favors long-running CPU-bound jobs may maximize throughput while making an interactive desktop unresponsive; a scheduler that aggressively favors latency-sensitive tasks may improve perceived responsiveness at the cost of throughput and fairness. The design of a general-purpose scheduler is therefore an exercise in constrained optimization under uncertain and heterogeneous workload conditions.

In Linux, scheduling is implemented as a hierarchy of **scheduling classes**. Each class encapsulates a policy, and the core dispatcher invokes classes in priority order until a runnable task is found[reference:0]. The **fair scheduling class** handles ordinary user tasks—those using `SCHED_NORMAL`, `SCHED_BATCH`, and `SCHED_IDLE`—and for more than fifteen years its principal algorithm was the **Completely Fair Scheduler (CFS)** . CFS was merged in Linux 2.6.23 and was designed by Ingo Molnar as a replacement for the earlier `SCHED_OTHER` scheduler[reference:1]. Its central goal was to approximate an "ideal, precise multi-tasking CPU" on real hardware[reference:2]. Understanding CFS therefore requires understanding both the elegant conceptual model that motivates it and the engineering compromises that make it practical.

1.2 Why Scheduling Matters in Multitasking Environments

Modern systems routinely run hundreds or thousands of threads. Even a modest desktop session may have a browser compositor, audio server, file indexer, compiler, and several background daemons competing for a handful of cores. The scheduler determines whether the user perceives smooth scrolling, whether an audio buffer underruns, whether a build completes promptly, and whether a background job can monopolize the machine. Because workloads are bursty and heterogeneous, no single static priority assignment can be optimal for all phases of execution. A scheduler must continuously re-evaluate which task should run, how long it should run, and on which CPU it should run. These decisions interact with cache locality, memory bandwidth, power management, and synchronization, which is why scheduling remains an active area of systems research despite decades of study.

1.3 Fairness, Throughput, Responsiveness, and Predictability

Four objectives are frequently used to evaluate a general-purpose scheduler, and they are often in tension:

- **Fairness** concerns how CPU time is distributed among runnable tasks. A fair scheduler attempts to give each task a share proportional to its entitlement, rather than allowing one task to monopolize the processor.

- **Throughput** measures the amount of useful work completed per unit time. Throughput-oriented policies may favor long CPU bursts and reduce context-switching overhead.
- **Responsiveness** measures how quickly a task reacts to an external event. Interactive and latency-sensitive workloads benefit from short scheduling delays, even if this reduces raw throughput.
- **Predictability** concerns the variance of response and completion times. Real-time systems require bounded worst-case latency, whereas general-purpose systems usually aim for low average latency and acceptable tail latency.

CFS is explicitly a *fairness*-oriented scheduler. It does not promise hard deadline guarantees, and its behavior should not be confused with that of a real-time scheduler. This distinction is developed in Chapters 9 and 10.

1.4 Historical Evolution of Linux CPU Scheduling

Linux scheduling has evolved through several distinct phases. Early kernels used a simple circular run-queue with a goodness function that combined priority and remaining timeslice. The **O(1) scheduler** , introduced in the 2.6 series, used per-CPU priority arrays and allowed constant-time selection, but its heuristics for interactivity became difficult to tune and produced inconsistent fairness. The **Rotating Staircase Deadline Scheduler (RSDL)** and its predecessor **SD** explored fair-share ideas before CFS was merged in 2.6.23. CFS replaced the notion of a fixed timeslice with a virtual clock and a red-black tree, making fairness accounting explicit and removing many ad hoc heuristics[reference:3]. In Linux 6.6, released in October 2023, the fair class began transitioning to **EEVDF** , an algorithm originally proposed by Stoica and Abdel-Wahab in 1995 and adapted for Linux by Peter Zijlstra[reference:4][reference:5]. This transition is important: statements about "the Linux scheduler" that were accurate for kernel 5.x may be inaccurate for kernel 6.6 and later.

1.5 Motivation Behind CFS

The immediate motivation for CFS was the perceived unfairness and heuristic complexity of the O(1) scheduler, particularly under mixed interactive and CPU-bound workloads. The O(1) scheduler attempted to infer whether a task was interactive by observing sleep behavior and adjusting its priority accordingly. These heuristics were fragile: a task could be misclassified, and the resulting priority adjustments could produce large fairness deviations. CFS took a different approach. Rather than estimating interactivity, it models an ideal processor that can run all runnable tasks simultaneously at equal speed, and then tracks how much virtual time each task has consumed. The task with the smallest virtual runtime is selected next. This single rule provides a coherent fairness criterion, naturally accommodates priority through weighting, and avoids many of the pathologies of earlier heuristics[reference:6].

1.6 Research Objectives, Scope, and Methodology

This paper has five objectives: (i) to explain the conceptual foundations of CFS, including ideal multitasking, virtual runtime, weights, and the red-black tree; (ii) to describe the algorithm workflow, pseudocode, and a worked example; (iii) to analyze multiprocessor and multicore scheduling, including load balancing, affinity, and NUMA considerations; (iv) to compare CFS with the classical real-time algorithms RM and EDF; and (v) to identify the limitations of CFS and the significance of the CFS-to-EEVDF transition. The methodology is analytical and documentary. It draws on official Linux kernel

documentation, kernel source documentation, foundational scheduling literature, and authoritative man pages. Quantitative examples are constructed for pedagogical purposes and are explicitly labeled as illustrative; no claim is made that they reproduce benchmark measurements from a specific kernel build.

1.7 Organization of the Paper

Chapter 2 reviews scheduling fundamentals and metrics. Chapter 3 presents the CFS model in detail, including its data structures and its relationship to scheduling classes. Chapter 4 describes the algorithm workflow and provides a flowchart. Chapter 5 gives language-independent pseudocode and a complete worked example. Chapter 6 evaluates the main features and advantages of CFS. Chapter 7 examines multiprocessing and multicore scheduling. Chapter 8 presents illustrative charts and explains their assumptions. Chapter 9 compares CFS, RM, and EDF. Chapter 10 discusses limitations. Chapter 11 treats starvation, deadlock, and priority inversion. Chapter 12 addresses security, reliability, and practical configuration. Chapter 13 provides a reproducible Linux demonstration. Chapter 14 discusses future directions, and Chapter 15 concludes.

2. Fundamentals of CPU Scheduling

2.1 Processes, Threads, Tasks, and CPU Bursts

In Linux, the scheduler operates on **tasks** (represented by `struct task_struct`), which may correspond to processes or threads depending on the sharing of address space and other resources. A task alternates between **CPU bursts** and **I/O bursts**. CPU-bound tasks have long CPU bursts and infrequent I/O; I/O-bound and interactive tasks have short CPU bursts and frequent blocking. The distribution of burst lengths is workload-dependent and often heavy-tailed. A scheduler cannot know future burst lengths, so it must make decisions online using observed behavior and declared parameters such as nice value and scheduling policy. CFS does not classify tasks as interactive or CPU-bound; instead, it accounts for actual execution through virtual runtime, which allows it to respond to observed demand rather than inferred intent[reference:7].

2.2 Preemptive versus Non-Preemptive Scheduling

A **non-preemptive** scheduler allows a running task to continue until it voluntarily blocks, terminates, or yields the CPU. A **preemptive** scheduler can interrupt a running task when a higher-priority task becomes runnable or when its time slice expires. Preemption is essential for responsiveness and fairness in general-purpose systems, but it introduces context-switching overhead and can disrupt cache locality. Linux is preemptive in the kernel (with some exceptions in atomic sections) and CFS relies on preemption to enforce fair sharing. Table 2.1 summarizes the trade-offs.

Table 2.1 — Comparison of preemptive and non-preemptive scheduling.

CRITERION	PREEMPTIVE	NON-PREEMPTIVE
Decision points	Arrival, wake-up, tick, blocking, yield	Blocking, termination, voluntary yield
Responsiveness	Typically better; a newly runnable high-priority task can preempt	Potentially poor; a long-running task can delay others
Fairness	Easier to enforce with time slicing	Requires cooperation; unfair to short tasks
Overhead	Higher: context switches, scheduler invocations	Lower: fewer switches
Cache behavior	More disruption due to migrations and switches	Better locality for long-running tasks
Typical use	General-purpose OS, real-time OS	Batch systems, some embedded systems

2.3 Scheduling Queues and Context Switching

Runnable tasks are organized into **run queues** . In CFS, each CPU has a `cfs_rq` structure that contains a red-black tree of runnable scheduling entities ordered by `vruntime` [reference:8]. When the scheduler switches from one task to another, it performs a **context switch** : it saves the register state of the outgoing task and restores the state of the incoming task. Context switches are not free; they consume cycles, pollute caches and translation lookaside buffers (TLBs), and can interact poorly with simultaneous multithreading. CFS attempts to reduce unnecessary switches by using a wake-up gran-

ularity that prevents a newly woken task from immediately preempting the current task unless it has a sufficiently smaller virtual runtime[reference:9]. This is a deliberate trade-off between fairness and overhead.

2.4 Performance Metrics and Worked Examples

Scheduling decisions are evaluated using standard metrics. Let n be the number of tasks, T_i the completion time of task i, A_i its arrival time, B_i its total CPU burst requirement, and W_i its waiting time. Then:

$$Turnaround_i = T_i - A_i \tag{2.1}$$

$$Waiting_i = Turnaround_i - B_i \tag{2.2}$$

$$Response_i = FirstRun_i - A_i \tag{2.3}$$

$$Throughput = \frac{\text{number of completed tasks}}{\text{total elapsed time}} \tag{2.4}$$

$$CPU\ utilization = \frac{\text{busy time}}{\text{total time}} \tag{2.5}$$

As a simple illustration, consider three tasks that arrive at time 0 with burst requirements $B_1 = 8$, $B_2 = 4$, and $B_3 = 2$ time units. Under a non-preemptive first-come-first-served policy in the order 1, 2, 3, the completion times are 8, 12, and 14, so the average turnaround time is (8 + 12 + 14)/3 = 11.33 and the average waiting time is (0 + 8 + 12)/3 = 6.67. If task 3 is served first, then task 2, then task 1, the completion times are 2, 6, and 14, the average turnaround time is 7.33, and the average waiting time is 2.67. This classic example demonstrates that scheduling order materially affects user-visible latency even when total work is unchanged. CFS does not optimize average turnaround time directly; its objective is proportional fairness, but the example illustrates why scheduling policy matters.

Table 2.2 — Scheduling metrics and their definitions.

METRIC	DEFINITION	PRIMARY CONCERN
CPU utilization	Fraction of time the CPU is doing useful work	Efficiency
Throughput	Completed tasks per unit time	Efficiency
Turnaround time	Completion time minus arrival time	Batch performance
Waiting time	Time spent in the run queue	Fairness / batch
Response time	Time from arrival to first execution	Interactivity
Fairness	Deviation from entitled CPU share	Proportional sharing
Overhead	Time spent in scheduler and context switches	Efficiency

2.5 Interactive versus CPU-Intensive Workloads

Interactive workloads—text editors, terminals, browsers, audio servers—typically exhibit short CPU bursts followed by blocking on I/O or user input. Their perceived performance depends heavily on **response time** and **scheduling latency** . CPU-intensive workloads—compilers, video encoders, scientific simulations—have long CPU bursts and benefit from **throughput** and cache locality. A general-purpose scheduler must serve both classes without explicit classification. CFS's virtual-runtime accounting naturally gives a task that has slept a long time a relatively small `vruntime` (subject to placement rules), so it can be scheduled promptly when it wakes, while a CPU-bound task that has consumed a great deal of CPU accumulates virtual runtime and yields to others[reference:10]. This is not a hard latency guarantee, but it is a principled way to balance the two workload types.

2.6 General-Purpose versus Real-Time Scheduling

General-purpose scheduling aims to provide good average-case behavior across diverse workloads. Real-time scheduling aims to guarantee that every task meets its deadline, even in the worst case. These objectives are different and can conflict. A hard real-time scheduler may sacrifice fairness and throughput to meet deadlines; a fair scheduler may delay a deadline-sensitive task in favor of fairness. Linux exposes both worlds through scheduling classes. The fair class (CFS, and later EEVDF) handles ordinary tasks. The real-time class implements `SCHED_FIFO` and `SCHED_RR` with priorities 1–99, and the deadline class implements `SCHED_DEADLINE` based on EDF with admission control[reference:11]. Real-time tasks always preempt fair-class tasks. CFS itself does not provide hard real-time guarantees, and this distinction is essential when comparing CFS with RM and EDF in Chapter 9.

2.7 SMP and Multicore Scheduling Fundamentals

On a symmetric multiprocessing (SMP) system, each CPU has its own run queue, and the scheduler must decide not only which task to run but also on which CPU to run it. This introduces **load balancing** , **task migration** , **CPU affinity** , and **cache locality** as first-class concerns. Migrating a task to an idle CPU may improve utilization but can destroy cache and TLB state. Linux models the CPU topol-

ogy as a hierarchy of **scheduling domains** built from cache and NUMA boundaries, and load balancing attempts to move work within domains before moving it across domains[reference:12]. These issues are examined in detail in Chapter 7.

3. The Completely Fair Scheduler (CFS)

3.1 Ideal Multitasking

The central conceptual model of CFS is the **ideal, precise multi-tasking CPU** . Imagine a processor with 100% power that can execute all runnable tasks simultaneously, each at speed *1/n*, where *n* is the number of runnable tasks. If two tasks are runnable, each receives 50% of the processor; if four are runnable, each receives 25%. This ideal machine has no scheduling latency and no context-switching overhead. Real hardware can execute only a limited number of tasks at a time, so CFS approximates the ideal by tracking how much virtual time each task would have received on the ideal machine and by always running the task that is furthest behind[reference:13]. The approximation is the essence of the design: 80% of CFS's design, as the kernel documentation puts it, can be summarized in the sentence that CFS models an ideal multi-tasking CPU on real hardware[reference:14].

3.2 Proportional Fairness

Pure equal sharing is not sufficient for a general-purpose system because users and administrators need to assign different weights to tasks. CFS therefore implements **proportional fairness** : each task is entitled to a share of CPU time proportional to its weight. If tasks *A* and *B* have weights w_A and w_B, then in a saturated system the long-run CPU shares should satisfy:

$$\frac{CPU_{\mathrm{A}}}{CPU_{\mathrm{B}}} \approx \frac{w_{\mathrm{A}}}{w_{\mathrm{B}}} \tag{3.1}$$

This is a statement about long-run proportions under sustained contention. It is not a guarantee about any short interval, and it assumes that all tasks are runnable for the entire interval. In practice, tasks block, wake, migrate, and change weight, so the achieved shares are approximations. Group scheduling extends the same idea hierarchically: a control group can be treated as a scheduling entity with its own weight, and tasks within the group share the group's allocation[reference:15].

3.3 Virtual Runtime (`vruntime`)

CFS tracks, for every scheduling entity, a **virtual runtime** stored in `p→se.vruntime` (or the corresponding field of a group scheduling entity) in nanosecond units[reference:16]. Virtual runtime is not wall-clock runtime. It is a normalized measure of how much CPU service the entity has received relative to its weight. When an entity executes for Δ_{exec} nanoseconds, its virtual runtime advances by an amount that is inversely proportional to its weight:

$$\Delta vruntime = \Delta_{exec} \times \frac{W_{\mathrm{o}}}{w_{\mathrm{i}}} \tag{3.2}$$

Here W_0 is the weight of a reference task (the weight corresponding to nice value 0, conventionally 1024 in classic CFS) and w_i is the weight of entity i. A task with a larger weight accumulates virtual runtime more slowly, so it can execute longer before its virtual runtime catches up with that of a lower-weight task. Conversely, a task with a smaller weight accumulates virtual runtime quickly and will be preempted sooner. The scheduler selects the entity with the smallest virtual runtime, which is the entity that has received the least service relative to its entitlement[reference:17]. Figure 3.1 illustrates the conceptual relationship.

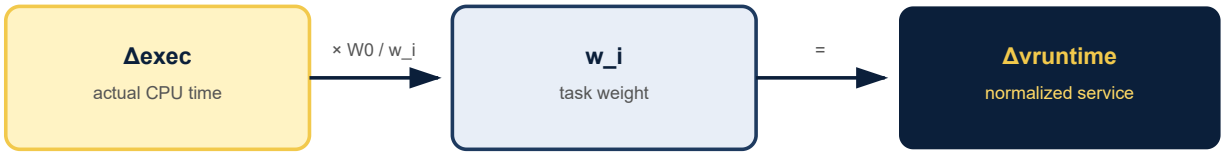


Figure 3.1 — Conceptual relationship between actual runtime, task weight, and virtual runtime. A larger weight reduces the rate at which virtual runtime accumulates. The equation is a simplified educational model; production kernels use additional adjustments for group scheduling, lag, and placement.

3.4 Task Weights and Scheduling Priority

In classic CFS, each task's weight is derived from its **nice value** , an integer in the range −20 to +19. Nice value 0 is the default and has weight 1024. Each step of one nice level changes the weight by a factor of approximately 1.25, which corresponds to roughly a 10% change in CPU share under contention[reference:18]. The exact mapping is a lookup table in the kernel. Table 3.1 summarizes the key CFS concepts and Table 3.2 gives illustrative weights for selected nice values.

Table 3.1 — Major CFS concepts and terminology.

CONCEPT	MEANING IN CFS
Ideal multi-tasking CPU	A conceptual processor that runs all runnable tasks simultaneously at 1/n speed
`vruntime`	Normalized virtual runtime; the primary scheduling key
Weight	Value derived from nice level; determines the rate of `vruntime` accumulation
Scheduling entity	A task or a group of tasks that can be scheduled as a unit
Run queue (`cfs_rq`)	Per-CPU structure holding runnable entities
Red-black tree	Self-balancing tree ordered by `vruntime`; leftmost node is selected next
`min_vruntime`	Monotonically increasing lower bound used to place new or woken entities
Wake-up granularity	Threshold that prevents a newly woken task from preempting immediately
Group scheduling	Hierarchical fair sharing through control groups
Load balancing	Migration of runnable entities across CPUs

Table 3.2 — Illustrative nice values, weights, and relative CPU shares (classic CFS weight table; approximate).

NICE VALUE	WEIGHT	SHARE RELATIVE TO NICE 0	QUALITATIVE MEANING
−20	88,761	≈ 86.7×	Highest conventional priority
−10	9,543	≈ 9.3×	High priority
−5	3,350	≈ 3.3×	Moderately elevated priority
0	1,024	1.0×	Default
+5	335	≈ 0.33×	Moderately reduced priority
+10	110	≈ 0.11×	Low priority
+19	15	≈ 0.015×	Lowest conventional priority

3.5 Nice Values and CPU Allocation

The nice value is a *relative* hint, not an absolute reservation. If two tasks with nice values 0 and 5 are the only runnable tasks, their weight ratio is approximately 1024:335, so the nice-0 task should receive about 75% of the CPU and the nice-5 task about 25% under sustained contention. If a third task with nice −5 joins, the proportions change. Nice values also do not override scheduling classes: a real-time task with `SCHED_FIFO` always preempts a fair-class task regardless of nice value. Changing a task's nice value is therefore different from changing its scheduling policy, a point emphasized in Chapter 13[reference:19].

3.6 Scheduling Entities

CFS schedules **scheduling entities** rather than raw tasks. A scheduling entity is represented by `struct sched_entity` and may correspond to an individual task or to a task group. This abstraction allows CFS to treat a group of tasks as a single weighted entity, enabling hierarchical fairness. Each entity has its own `vruntime`, weight, and load contribution. When a group is selected, the scheduler recursively selects a task within the group using the same fair algorithm. The kernel documentation notes that CFS schedules `sched_entity` structures, not tasks directly, which allows groups of tasks to be scheduled similarly to single tasks[reference:20]. Group scheduling is discussed further in Section 3.13.

3.7 The Runnable-Task Structure and Red-Black Tree

Classic CFS stores runnable scheduling entities in a **red-black tree** keyed by `vruntime` [reference:21]. A red-black tree is a self-balancing binary search tree that guarantees O(log n) insertion, deletion, and lookup, while allowing the minimum element to be found in O(1) time by following left-child pointers. CFS maintains the invariant that the leftmost node is the entity with the smallest virtual runtime, i.e., the next entity to run. The run queue also maintains `min_vruntime`, a monotonically increasing value that tracks the smallest virtual runtime observed on that run queue. When a task is created or wakes from sleep, its virtual runtime is placed relative to `min_vruntime` so that it does not unfairly jump to the front of the tree after a long sleep[reference:22][reference:23]. Figure 3.2 shows a simplified red-black tree.

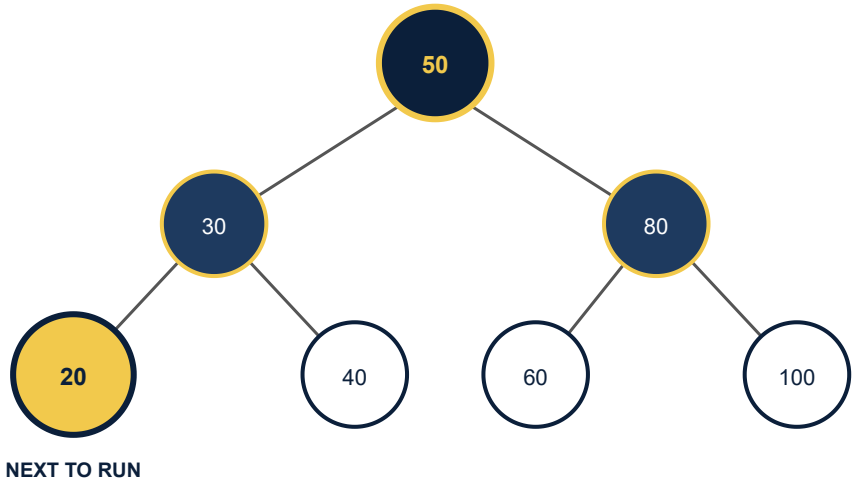


Figure 3.2 — Simplified red-black tree of runnable scheduling entities keyed by `vruntime`. The leftmost node (20) is the next entity selected in classic CFS. Real trees may contain group entities and are rebalanced after insertions and deletions.

3.8 Choosing the Next Task

The selection rule in classic CFS is simple: choose the leftmost entity in the red-black tree, i.e., the entity with the smallest `vruntime` [reference:24]. If the tree is empty, the CPU runs the idle task. Because the tree is ordered by virtual runtime, the selection step is O(1) after the tree has been main-

tained. The complexity lies in maintaining the tree, updating virtual runtime, and deciding when to preempt. In EEVDF, selection is more nuanced: the scheduler considers *lag* (whether an entity is owed or has exceeded service) and *virtual deadline* (the time by which the entity should receive its requested slice), and it may choose among eligible entities with non-negative lag the one with the earliest virtual deadline[reference:25]. This is discussed in Section 3.15.

3.9 Updating Virtual Runtime

Virtual runtime is updated during execution and at scheduling events. When an entity runs for Δ_{exec}, its virtual runtime increases according to Equation 3.2. In addition, the scheduler maintains `min_vruntime` and may adjust the virtual runtime of a newly woken entity so that it is placed near the left of the tree without giving it an unfair advantage. The exact update logic in production kernels is more complex than Equation 3.2: it accounts for group scheduling, load tracking (PELT), and, in newer kernels, lag and virtual deadline computations. Equation 3.2 should therefore be understood as a conceptual model that explains the direction and approximate magnitude of the effect, not as a line-by-line description of `kernel/sched/fair.c` [reference:26].

3.10 Preemption and Scheduling Decisions

A scheduling decision occurs when a task blocks, terminates, yields, wakes, or when the scheduler tick detects that the running task has exhausted its fair share or that a newly runnable task has a sufficiently smaller virtual runtime. CFS uses a **wake-up granularity** to avoid excessive preemption: a waking task does not necessarily preempt the current task immediately unless the difference in virtual runtime exceeds a threshold. This reduces context switches and preserves cache locality[reference:27]. Preemption is also influenced by the scheduling class hierarchy: a real-time task waking up will preempt a fair-class task immediately, subject to real-time throttling if configured[reference:28].

3.11 Task Creation, Termination, Blocking, and Waking

When a task is created, it is initialized with a weight derived from its nice value, a virtual runtime, and a scheduling entity that is inserted into the appropriate run queue. A newly created task is typically placed so that it does not obtain an excessive share relative to existing tasks. When a task blocks, it is removed from the runnable tree and its state is saved. When it wakes, it is reinserted with a virtual runtime adjusted relative to `min_vruntime`; this prevents a long-sleeping task from returning with an artificially small virtual runtime and monopolizing the CPU[reference:29]. When a task terminates, it is removed from the tree and its resources are released. These operations must be synchronized with load balancing and with concurrent modifications on other CPUs.

3.12 Load Balancing and CPU Affinity

On an SMP system, CFS maintains a separate run queue per CPU. Load balancing attempts to keep the load across CPUs roughly equal while respecting **CPU affinity** (the set of CPUs on which a task is allowed to run) and **cache locality** . Migrating a task to another CPU can improve utilization but may incur cache and TLB misses. Linux organizes CPUs into **scheduling domains** that reflect shared

caches and NUMA topology, and the load balancer tries to move tasks within a domain before moving them across domains[reference:30]. Affinity is exposed to user space through `taskset` and `sched_setaffinity(2)`. Load balancing is discussed in detail in Chapter 7.

3.13 Group Scheduling and Hierarchical Fairness

Group scheduling extends fairness from individual tasks to groups of tasks, typically defined by control groups (cgroups). Each group is represented by a scheduling entity with its own weight and virtual runtime, and the group's children are scheduled within the group's allocation. This enables hierarchical fair sharing: a group with weight W_G receives a share of CPU time proportional to W_G relative to other groups, and within the group, tasks share the group's allocation according to their own weights[reference:31]. Group scheduling is particularly important in container environments, where multiple containers may need to be isolated from one another. The cgroup v2 CPU controller exposes bandwidth limits through `cpu.max` and weight through `cpu.weight`, and CFS bandwidth control can throttle a group that exceeds its quota[reference:32].

3.14 CFS and Linux Scheduling Classes

CFS is not the only scheduler in Linux; it is the implementation of the **fair scheduling class** . The scheduling class hierarchy determines the order in which policies are consulted. The stop class has the highest priority and is used internally for CPU migration and hotplug. The deadline class implements `SCHED_DEADLINE` based on EDF with admission control. The real-time class implements `SCHED_FIFO` and `SCHED_RR` with priorities 1–99. The fair class implements `SCHED_NORMAL`, `SCHED_BATCH`, and `SCHED_IDLE`. The idle class runs the per-CPU idle thread when no other task is runnable[reference:33]. This hierarchy means that a real-time task will always preempt a CFS task, and a deadline task will preempt a real-time task if its deadline requires it. CFS therefore operates within a bounded region of the scheduling policy space; it cannot override the guarantees of higher-priority classes.

3.15 Classic CFS versus EEVDF

Classic CFS selects the entity with the smallest virtual runtime. EEVDF, introduced as the fair-class algorithm in Linux 6.6, retains the idea of virtual runtime but adds two concepts: **lag** and **virtual deadline** . Lag measures whether an entity has received more or less than its fair share: a positive lag means the entity is owed CPU time, and a negative lag means it has exceeded its share. EEVDF considers only entities with non-negative lag (eligible entities) and, among them, selects the one with the earliest virtual deadline, which is derived from the entity's requested time slice[reference:34]. This allows latency-sensitive tasks to request shorter slices and receive earlier deadlines, improving responsiveness without abandoning fairness. EEVDF also replaces several CFS heuristics and tunables with algorithmic rules, and it introduces a decaying lag mechanism for sleeping tasks to prevent them from exploiting the system by sleeping briefly to reset their lag[reference:35]. The transition is gradual: Linux 6.6 introduced EEVDF as the fair-class implementation, and later kernels continued to refine it. Table 3.3 summarizes the differences.

Table 3.3 — Classic CFS versus newer EEVDF-related scheduling concepts.

ASPECT	CLASSIC CFS	EEVDF (LINUX 6.6+)
Primary selection key	Smallest `vruntime`	Earliest virtual deadline among eligible entities with non-negative lag
Fairness measure	Virtual runtime	Lag (owed or exceeded service)
Latency control	Heuristics and wake-up granularity	Requested slice and virtual deadline
Tunables	Several sysctls and heuristics	Fewer; algorithmic rules replace heuristics
Sleeping-task handling	Placement relative to `min_vruntime`	Decaying lag over virtual run time
Data structure	Red-black tree by `vruntime`	Red-black tree with lag and deadline keys
Kernel version	2.6.23 through 6.5 (as fair class)	6.6 and later (fair class)

VERSION NOTE

Statements in this paper that describe “CFS” refer to the classic fair-class algorithm unless explicitly marked otherwise. Linux 6.6 replaced the CFS selection logic with EEVDF in the fair class[reference:36] [reference:37]. The conceptual foundations—weights, virtual runtime, proportional fairness, group scheduling, and load balancing—remain relevant, but implementation details and selection behavior differ.

4. CFS Algorithm Workflow

4.1 Step-by-Step Workflow

The following sequence describes the conceptual workflow of classic CFS. It is intentionally generic; a specific kernel version may add, remove, or reorder steps.

1. **Task becomes runnable.** A task is created, wakes from sleep, or is unblocked. Its scheduling entity is initialized or updated with its weight and virtual runtime.
2. **Scheduling parameters are accounted for.** The entity's weight is derived from its nice value, and any group hierarchy is taken into account.
3. **The task enters the relevant run queue.** The entity is inserted into the per-CPU red-black tree keyed by `vruntime`. New or woken entities are placed relative to `min_vruntime` to avoid unfair advantage.
4. **The scheduler compares runnable entities.** The tree orders entities by virtual runtime, so the leftmost entity is the one with the smallest virtual runtime.
5. **A task is selected.** In classic CFS, the leftmost entity is chosen. In EEVDF, an eligible entity with the earliest virtual deadline is chosen.
6. **The selected task executes.** The CPU switches to the selected task, restoring its register state and address space as needed.
7. **Execution time and virtual runtime are updated.** The entity's actual runtime is accounted for, and its virtual runtime advances according to its weight.
8. **A scheduling decision occurs when required.** The running task may block, yield, be preempted by a higher-priority class, or be preempted because another fair-class entity has a sufficiently smaller virtual runtime.
9. **The scheduler selects another task or continues execution.** If the current task remains the best choice, it continues; otherwise, a context switch occurs.
10. **Load balancing may redistribute work.** Periodically or on events, the scheduler may migrate runnable entities across CPUs to balance load, subject to affinity and cache considerations.

4.2 Required Flowchart

Figure 4.1 presents the workflow as a flowchart. The diagram distinguishes decision points (diamonds) from process steps (rectangles) and shows the loop back to task selection after execution and accounting.

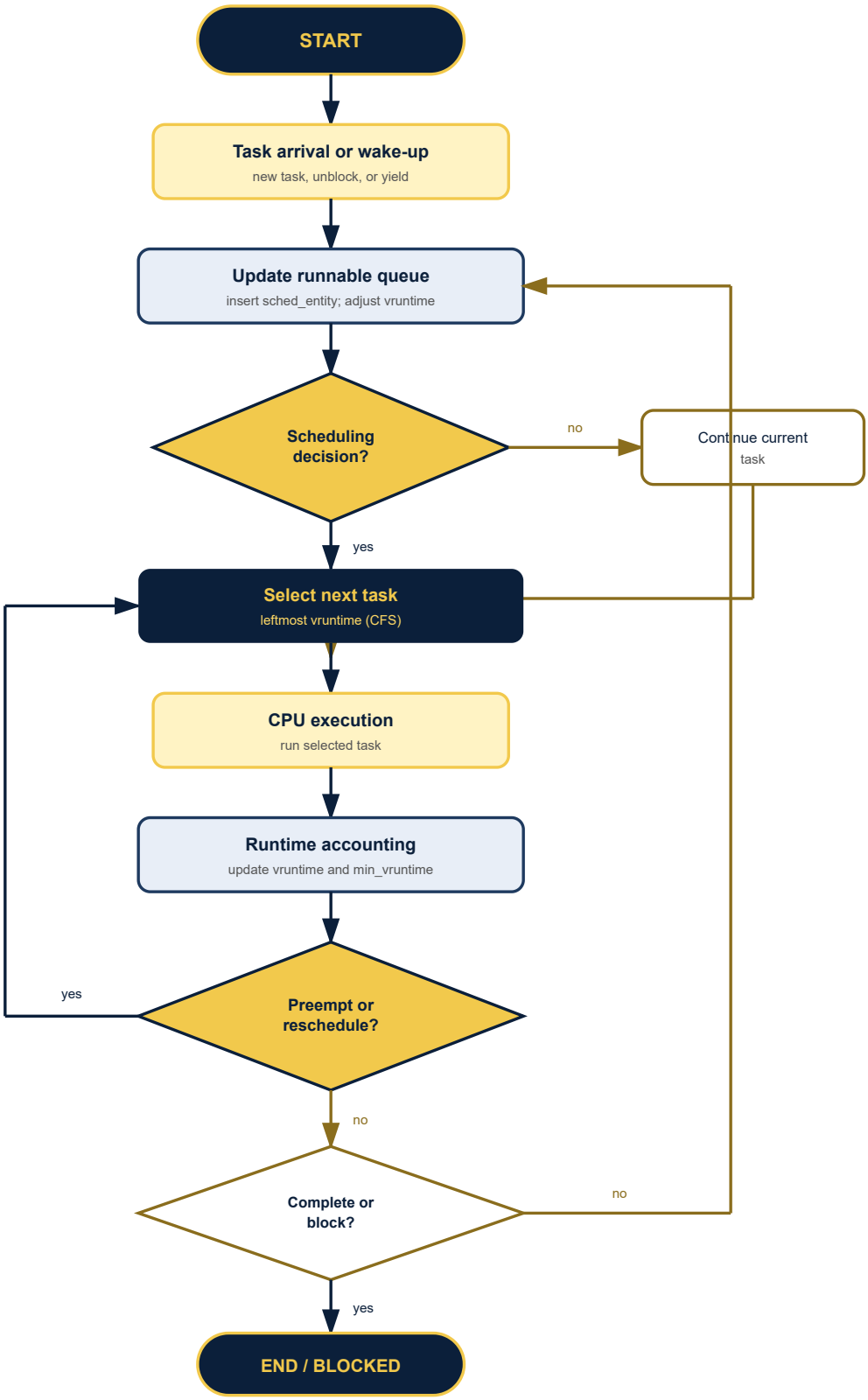


Figure 4.1 — CFS scheduling workflow flowchart. The loop from "runtime accounting" back to "select next task" represents the scheduler's continuous operation. The right-hand path shows that a task that neither blocks nor completes returns to the runnable queue and may be selected again. The diagram is conceptual and omits kernel-version-specific details such as EEVDF lag and virtual-deadline computations.

4.3 Conceptual Model versus Implementation

The workflow above is a teaching model. A production kernel implements many additional details: per-entity load tracking (PELT), hierarchical group scheduling, NUMA-aware load balancing, real-time throttling, and synchronization between CPUs. The selection rule in classic CFS is "leftmost `vruntime`," but the tree may contain group entities whose children are scheduled recursively. The accounting rule is "`vruntime` increases inversely with weight," but the kernel may adjust `vruntime` on wake-up, migration, and group changes. In EEVDF, the selection rule is "earliest virtual deadline among eligible entities with non-negative lag," which is a different algorithmic criterion even though it shares the same proportional-share goal. Readers should therefore treat the workflow as a conceptual scaffold and consult the kernel documentation for the specific version they are studying[reference:38][reference:39].

5. CFS Pseudocode and Worked Example

5.1 Conceptual Pseudocode

The following pseudocode is a language-independent educational model. It is not a copy of the Linux kernel implementation and omits synchronization, group scheduling, load balancing, and many edge cases. Its purpose is to make the conceptual algorithm explicit.

```
// Conceptual CFS educational model — not kernel source
// Each scheduling entity has:
//   id, weight (from nice), vruntime (virtual runtime), state

CONSTANT W0 = 1024          // reference weight at nice 0
CONSTANT MIN_GRANULARITY = 3_000_000  // illustrative ns
CONSTANT TARGET_LATENCY = 6_000_000   // illustrative ns

STRUCT Entity:
    id
    weight
    vruntime
    state        // RUNNABLE, RUNNING, BLOCKED, TERMINATED
    last_start

FUNCTION weight_from_nice(nice):
    // Lookup table; nice 0 → 1024, each step ~1.25x
    RETURN lookup_weight(nice)

FUNCTION init_entity(id, nice):
    e = new Entity
    e.id = id
    e.weight = weight_from_nice(nice)
    e.vruntime = 0
    e.state = RUNNABLE
    RETURN e

FUNCTION enqueue(runqueue, e):
    // Insert into red-black tree keyed by vruntime
    // New/woken entities are placed relative to min_vruntime
    IF e.is_new_or_woken:
        e.vruntime = max(e.vruntime, runqueue.min_vruntime)
    tree_insert(runqueue.rb_tree, e)

FUNCTION pick_next(runqueue):
    IF runqueue.is_empty():
        RETURN idle_task
    // Classic CFS: leftmost node has smallest vruntime
    e = tree_minimum(runqueue.rb_tree)
    RETURN e

FUNCTION update_vruntime(e, delta_exec):
    // Weighted virtual runtime increment
    e.vruntime = e.vruntime + delta_exec * (W0 / e.weight)

FUNCTION schedule(runqueue):
    current = runqueue.current
    next = pick_next(runqueue)

    IF next == current:
        RETURN  // continue execution
```

```
    IF current.state == RUNNING:
        current.state = RUNNABLE
        update_vruntime(current, now() - current.last_start)
        enqueue(runqueue, current)

    next.state = RUNNING
    next.last_start = now()
    context_switch(current, next)
    runqueue.current = next

FUNCTION on_tick(runqueue):
    current = runqueue.current
    delta = now() - current.last_start
    update_vruntime(current, delta)

    // Check whether another entity should run
    IF should_preempt(runqueue, current):
        schedule(runqueue)

FUNCTION should_preempt(runqueue, current):
    candidate = pick_next(runqueue)
    IF candidate == current:
        RETURN false
    // Wake-up granularity prevents excessive preemption
    RETURN (current.vruntime - candidate.vruntime) > wakeup_granularity()

FUNCTION on_block(e):
    e.state = BLOCKED
    tree_remove(e.runqueue.rb_tree, e)

FUNCTION on_wake(e, runqueue):
    e.state = RUNNABLE
    e.vruntime = max(e.vruntime, runqueue.min_vruntime)
    enqueue(runqueue, e)
    // May trigger preemption if e is sufficiently ahead

FUNCTION on_terminate(e):
    e.state = TERMINATED
    tree_remove(e.runqueue.rb_tree, e)

// Simplified multi-CPU note:
// Each CPU has its own runqueue. Load balancing periodically
// migrates runnable entities between runqueues to equalize load,
// subject to affinity and cache/NUMA constraints.
```

Listing 5.1 — Conceptual pseudocode for classic CFS. Illustrative constants and simplifications are used; production kernels differ substantially.

5.2 Worked Example with Three Tasks

Consider three CPU-bound tasks, *A*, *B*, and *C*, that become runnable at time 0 and remain runnable throughout the interval under study. Their nice values are 0, 5, and 10, respectively. Using the illustrative weights from Table 3.2, we have w_A = 1024, w_B = 335, and w_C = 110. The total weight is 1469. Under sustained contention, the ideal proportional shares are:

$$share_A = \frac{1024}{1469} \approx 69.7\% \tag{5.1}$$

$$share_B = \frac{335}{1469} \approx 22.8\% \tag{5.2}$$

$$share_C = \frac{110}{1469} \approx 7.5\% \tag{5.3}$$

Assume a simplified scheduling latency of 12 time units. The scheduler assigns a target time slice proportional to weight, subject to a minimum granularity. For illustration, suppose the first scheduling period is divided into slices of 7, 3, and 2 time units for A, B, and C, respectively. (The exact slice calculations in a real kernel depend on `sched_latency_ns`, `sched_min_granularity_ns`, and the number of runnable entities; the numbers here are chosen for arithmetic clarity.) Table 5.1 shows the initial parameters.

Table 5.1 — Initial task parameters for the worked example.

TASK	NICE	WEIGHT	IDEAL SHARE	FIRST SLICE
A	0	1024	69.7%	7
B	5	335	22.8%	3
C	10	110	7.5%	2

We track virtual runtime using Equation 3.2 with W_0 = 1024. For task A, $\Delta vruntime = \Delta exec \times 1024/1024 = \Delta exec$. For task B, $\Delta vruntime = \Delta exec \times 1024/335 \approx 3.06 \times \Delta exec$. For task C, $\Delta vruntime = \Delta exec \times 1024/110 \approx 9.31 \times \Delta exec$. The scheduler always selects the entity with the smallest virtual runtime. Table 5.2 traces the evolution over several intervals.

Table 5.2 — Virtual runtime evolution in the worked example (illustrative values).

STEP	EVENT	A VRUNTIME	B VRUNTIME	C VRUNTIME	SELECTED NEXT
0	All runnable at t=0	0	0	0	A (tie; lowest id)
1	A runs 7 units	7.00	0	0	C
2	C runs 2 units	7.00	0	18.62	B
3	B runs 3 units	7.00	9.18	18.62	A
4	A runs 3 units	10.00	9.18	18.62	B
5	B runs 1 unit	10.00	12.24	18.62	A
6	A runs 2 units	12.00	12.24	18.62	B
7	B runs 1 unit	12.00	15.30	18.62	A
8	A runs 3 units	15.00	15.30	18.62	B

Two observations follow. First, the task with the smallest virtual runtime is always selected; this is the operational meaning of "completely fair" in classic CFS. Second, the higher-weight task A accumulates virtual runtime more slowly, so it receives more CPU time over the long run. The example is deliberately simplified: it ignores wake-up granularity, group scheduling, and the fact that real tasks block and migrate. It also assumes that all three tasks remain runnable, which maximizes contention.

5.3 Expected versus Observed Allocation

Over a sufficiently long interval with all tasks continuously runnable, the observed CPU allocation should approach the ideal proportions in Equations 5.1–5.3. In the short trace above, the allocation is uneven because the scheduler alternates among tasks to equalize virtual runtime, and the trace ends before convergence. This is expected: proportional fairness is a long-run property, not a guarantee for every short window. Figure 5.1 shows a Gantt chart for the first few intervals.

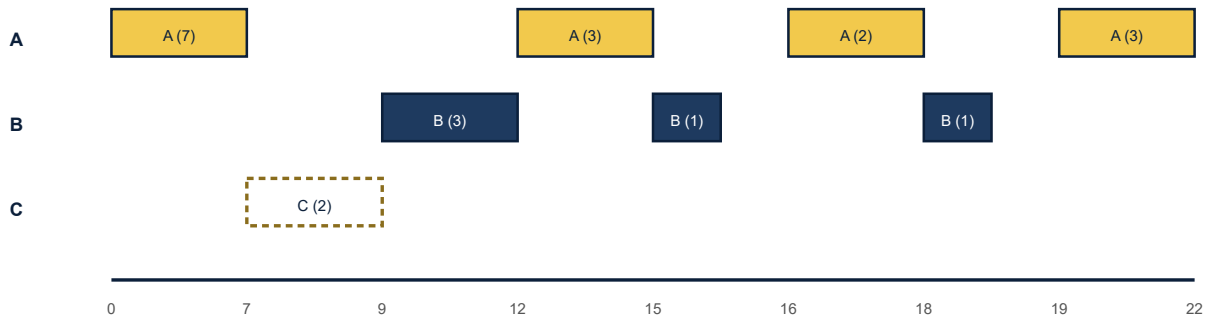


Figure 5.1 — Gantt chart for the worked three-task example. The vertical axis identifies tasks; the horizontal axis is time. The schedule is illustrative and is not drawn to scale. Real CFS schedules interleave more finely and use different slice lengths based on kernel tunables.

ASSUMPTION NOTE

All numerical values in this chapter are pedagogical. They are not measurements from a specific kernel version, CPU, or workload. Real CFS behavior depends on `sched_latency_ns`, `sched_min_granularity_ns`, `sched_wakeup_granularity_ns`, the number of runnable tasks, CPU topology, and cgroup configuration. Readers should not interpret the tables or Gantt chart as benchmark results[reference:40].

6. Main Features and Advantages of CFS

6.1 Proportional CPU Sharing

CFS provides a principled mechanism for proportional CPU sharing through weights derived from nice values. Unlike schedulers that use fixed time slices and ad hoc priority adjustments, CFS normalizes execution time by weight and uses a single virtual clock to compare entities. This makes the fairness objective explicit and auditable. The practical significance is that administrators can reason about relative CPU shares using weight ratios, and applications can request lower or higher priority through the standard nice interface. The limitation is that weights are relative, not reservations: if a high-weight task is blocked, its share is redistributed among runnable tasks, and there is no guarantee that a task will receive an absolute amount of CPU time.

6.2 Fairness Among Runnable Tasks

The defining feature of CFS is fairness among runnable tasks. Because the scheduler always selects the entity with the smallest virtual runtime, no runnable task can be indefinitely ignored as long as it remains runnable and the scheduler operates normally. This reduces the risk of persistent CPU starvation among fair-class tasks, although it does not eliminate starvation caused by higher-priority scheduling classes, cgroup throttling, or resource deadlocks (Chapter 11). Fairness is achieved without requiring interactive/CPU-bound classification, which makes CFS more robust to workload changes than heuristic schedulers.

6.3 Support for Different Task Priorities

CFS supports relative priorities through nice values and, in group scheduling, through control-group weights. This allows a server to give a database process a larger share than a backup job, or a desktop to give an audio thread a smaller nice value than a file indexer. The priority mechanism is smooth: each nice step changes the weight by approximately 1.25×, so users can fine-tune shares. However, nice values do not provide hard priority: a nice −20 task can still be preempted by a real-time task, and a low-nice task still receives some CPU time under contention. This is intentional; CFS is a fair scheduler, not a strict-priority scheduler.

6.4 Responsiveness Under Mixed Workloads

CFS's virtual-runtime accounting naturally accommodates interactive tasks. A task that has been sleeping has a relatively small virtual runtime when it wakes (subject to placement rules), so it can be scheduled promptly without being explicitly classified as interactive. The wake-up granularity prevents excessive preemption by newly woken tasks, balancing responsiveness against context-switching overhead. In practice, CFS improved desktop responsiveness relative to the O(1) scheduler, although latency-sensitive workloads can still suffer under heavy CPU contention. EEVDF further improves latency control by allowing tasks to request shorter time slices and by using virtual deadlines[reference:41].

6.5 Efficient Runnable-Task Management

The red-black tree provides O(log n) insertion and deletion and O(1) minimum lookup in classic CFS. This is efficient for typical run-queue sizes and scales well to thousands of runnable entities. The tree is self-balancing, so worst-case behavior is bounded. The run queue also maintains `min_vruntime` and load aggregates, which support placement and load balancing. The limitation is that the tree must be protected by locks on each CPU, and contention can become a bottleneck in extreme workloads with many short-lived tasks or frequent migrations.

6.6 Scalability Considerations

CFS uses per-CPU run queues, which reduces lock contention compared with a single global run queue. Each CPU schedules independently, and load balancing operates across CPUs. This design scales to large multicore systems, but scalability is not unlimited: cache-coherence traffic, migration costs, and NUMA effects can degrade performance. The kernel's scheduling-domain hierarchy and per-entity load tracking are designed to manage these trade-offs. The practical lesson is that CFS scalability depends on topology, workload, and configuration, not merely on the data structure.

6.7 SMP Support

CFS supports symmetric multiprocessing through per-CPU run queues, scheduling domains, and load balancing. Each CPU can make local scheduling decisions without consulting a global lock, and the load balancer periodically moves work to idle CPUs. CPU affinity allows administrators to pin tasks to specific CPUs for cache locality or isolation. These features make CFS suitable for servers, desktops, and embedded multicore systems. The trade-off between fairness and migration overhead is discussed in Chapter 7.

6.8 Load Balancing

Load balancing is essential for utilizing multiple CPUs. CFS's load balancer considers the load of each run queue, the topology of scheduling domains, and the affinity of tasks. It attempts to move tasks from busy CPUs to idle CPUs while respecting cache and NUMA locality. The balancer is not perfectly fair at every instant; it operates periodically and uses heuristics to avoid excessive migration. This is a deliberate compromise: perfect instantaneous balance would require frequent migrations and could destroy cache locality.

6.9 CPU Affinity

CPU affinity constrains the set of CPUs on which a task may run. It is useful for isolating latency-sensitive tasks, improving cache locality, and implementing CPU partitioning. CFS respects affinity during load balancing, although it may still migrate a task if the affinity mask allows it and the load imbalance is significant. Affinity is exposed through `taskset` and `sched_setaffinity(2)`. The limitation is that overly restrictive affinity can leave CPUs idle while others are overloaded, so affinity should be used with care.

6.10 Group Scheduling

Group scheduling extends fairness to hierarchies of tasks, which is essential for containers and multi-tenant systems. A control group can be assigned a weight or a bandwidth limit, and CFS enforces the group's share while scheduling its children fairly. This provides a degree of isolation between workloads and allows administrators to reason about resource allocation at the group level. The complexity is that hierarchical accounting and throttling interact in non-trivial ways, and misconfiguration can lead to unexpected throttling or unfairness[reference:42][reference:43].

6.11 Integration with Linux Scheduling Classes

CFS is integrated into Linux's scheduling-class hierarchy, which allows it to coexist with real-time and deadline scheduling. Real-time tasks preempt fair-class tasks, and deadline tasks have higher priority than real-time tasks. This layered design lets a single kernel serve general-purpose and real-time workloads, provided that real-time tasks are configured and throttled appropriately. CFS does not need to implement deadline guarantees itself; it delegates that responsibility to the deadline and real-time classes.

6.12 Practical Benefits and Limitations

Table 6.1 summarizes the principal advantages and disadvantages of CFS as a general-purpose scheduler.

Table 6.1 — Advantages and disadvantages of CFS.

ADVANTAGES	DISADVANTAGES / LIMITATIONS
Principled proportional fairness through virtual runtime and weights	No hard real-time deadline guarantees
No need to classify tasks as interactive or CPU-bound	Short-term fairness can deviate from long-run proportions
Efficient O(log n) run-queue operations	Per-CPU lock contention under extreme workloads
Good scalability through per-CPU run queues	Migration and cache/NUMA overhead
Support for nice values and group scheduling	Group scheduling and bandwidth control add complexity
Integration with real-time and deadline classes	Behavior depends on kernel version and configuration
Improved desktop responsiveness relative to older schedulers	Latency-sensitive workloads may still need real-time or EEVDF features

7. Multiprocessing and Multicore Scheduling

7.1 Symmetric Multiprocessing (SMP)

In an SMP system, all CPUs share the same memory and are treated as peers by the operating system. Each CPU can run any task, subject to affinity constraints. The scheduler must decide not only which task to run but also on which CPU to run it. This requires per-CPU run queues, inter-CPU synchronization, and a load-balancing mechanism. SMP is the dominant model for servers, desktops, and mobile devices, although heterogeneous (big.LITTLE) and NUMA systems add further complexity.

7.2 Multiple CPU Cores and Hardware Threads

Modern processors expose multiple physical cores and, often, simultaneous multithreading (SMT) hardware threads. Hardware threads share execution units and caches, so they are not equivalent to independent cores. The scheduler must account for the topology: sibling hardware threads, shared last-level caches, and NUMA nodes. Linux's scheduling domains encode this topology, and load balancing attempts to place tasks in a way that respects cache sharing and avoids unnecessary cross-node traffic[reference:44].

7.3 Per-CPU Run Queues

Each CPU has its own `cfs_rq` and its own red-black tree of runnable entities. Local scheduling decisions can be made without acquiring a global lock, which improves scalability. However, per-CPU queues can become imbalanced: one CPU may have many runnable tasks while another is idle. Load balancing is responsible for correcting this imbalance, subject to affinity and locality constraints.

7.4 Load Balancing Between CPUs

Load balancing operates at multiple levels. Periodic balancing runs at a configured interval and attempts to equalize load across CPUs within a scheduling domain. Event-driven balancing may occur when a CPU becomes idle, when a task wakes, or when a task changes its affinity. The balancer considers the load of each run queue, the number of runnable entities, and the cost of migration. It may pull tasks from busy CPUs or push tasks to idle CPUs. The goal is to keep all CPUs reasonably utilized without excessive migration.

7.5 Task Migration

Task migration moves a runnable task from one CPU's run queue to another's. Migration can improve utilization and reduce latency, but it has costs: the task's working set may no longer be in the destination CPU's cache, TLB entries may be invalidated, and the task may need to reacquire locks. CFS attempts to balance these costs against the benefits of load distribution. Migration is also constrained by CPU affinity and by the scheduling-domain hierarchy.

7.6 CPU Affinity and Cache Locality

CPU affinity restricts the set of CPUs on which a task may run. It is used to improve cache locality (by keeping a task on the CPU where its data is cached), to isolate latency-sensitive tasks, and to implement CPU partitioning. Affinity can be soft (a hint) or hard (a mask enforced by the scheduler). CFS respects hard affinity masks and may migrate tasks within the allowed set. Overly restrictive affinity can cause load imbalance, so it should be used judiciously.

7.7 Scheduling-Domain Concepts

A scheduling domain is a group of CPUs that share a scheduling property, such as a last-level cache or a NUMA node. Domains are organized hierarchically: the lowest level may contain SMT siblings, the next level may contain cores sharing a cache, and the highest level may span NUMA nodes. Load balancing attempts to balance within the lowest domain first and moves to higher domains only when necessary. This hierarchy reduces the cost of migration by preferring nearby CPUs[reference:45].

7.8 NUMA-Related Considerations

On a NUMA system, memory access latency depends on the physical location of memory relative to the CPU. Migrating a task to a CPU on a different NUMA node can improve CPU utilization but may increase memory latency if the task's data remains on the original node. CFS's NUMA-aware load balancing considers both CPU load and memory locality. The trade-off is complex, and the optimal policy depends on the workload's memory access patterns and the system's interconnect topology.

7.9 Fairness versus Migration Overhead

Perfect load balance at every instant would require frequent migrations, which would destroy cache locality and increase overhead. CFS therefore balances periodically and uses hysteresis to avoid moving tasks unnecessarily. This means that short-term load imbalance is accepted in exchange for lower migration overhead and better cache behavior. The balance is tunable to some extent, but the fundamental trade-off is inherent to multicore scheduling.

7.10 Synchronization and Contention

Per-CPU run queues require synchronization when tasks migrate, when load balancing reads remote queue state, and when scheduling parameters change. Lock contention can become a bottleneck in workloads with many short-lived tasks or frequent migrations. The kernel uses per-CPU locks, read-copy-update (RCU), and careful data structuring to reduce contention. Even so, extreme workloads can expose scalability limits, which is one motivation for ongoing scheduler research.

7.11 Multiprocessor Diagram

Figure 7.1 illustrates four CPUs, each with its own run queue, and possible task migrations. CPU 0 is heavily loaded, CPU 3 is idle, and the load balancer may migrate a task from CPU 0 to CPU 3, subject to affinity and cache considerations.

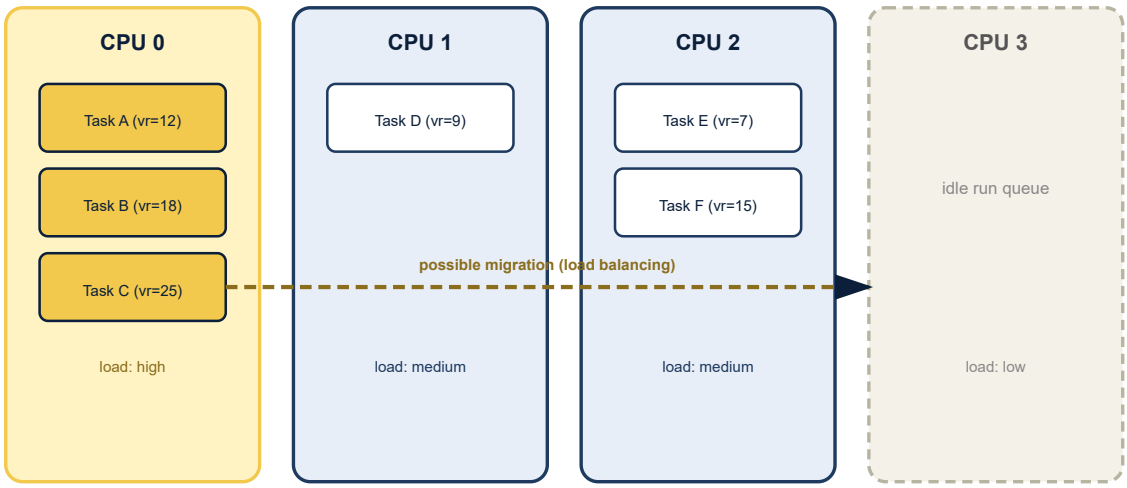


Figure 7.1 — Per-CPU run queues and possible task migration in an SMP system. The dashed arrow indicates a candidate migration from CPU 0 to CPU 3. Real load balancing considers scheduling domains, cache affinity, and NUMA topology, and it does not necessarily migrate the task shown.

8. Performance Evaluation and Graphs

This chapter presents illustrative charts that explain qualitative and quantitative properties of CFS. **All values are simulated or pedagogical, not benchmark measurements.** The charts are intended to help readers reason about scheduling behavior; they should not be cited as evidence about the performance of a particular kernel on a particular workload.

8.1 Illustrative CPU Allocation

Figure 8.1 compares the ideal long-run CPU shares of four tasks with nice values −5, 0, 5, and 10, assuming all are continuously runnable and ignoring group scheduling. The bars show the ideal proportions computed from the weight table, not measured allocation. The purpose is to illustrate the effect of nice values on entitlement.

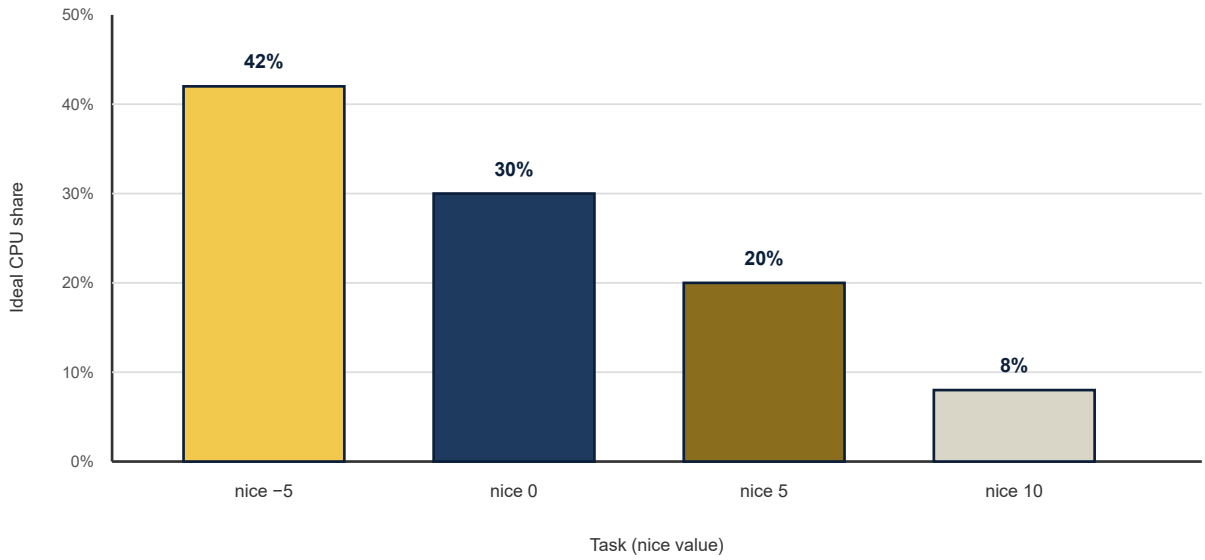


Figure 8.1 — Illustrative ideal CPU allocation among four continuously runnable tasks with different nice values. The values are pedagogical approximations derived from weight ratios, not measurements from a running system. Real allocations vary with workload, kernel version, and cgroup configuration.

8.2 Virtual Runtime Evolution

Figure 8.2 traces the virtual runtime of three tasks under the simplified model of Chapter 5. Task A has nice 0, task B has nice 5, and task C has nice 10. The slopes differ because virtual runtime accumulates more slowly for higher-weight tasks. The scheduler selects the task with the smallest virtual runtime at each step, producing the characteristic interleaving.

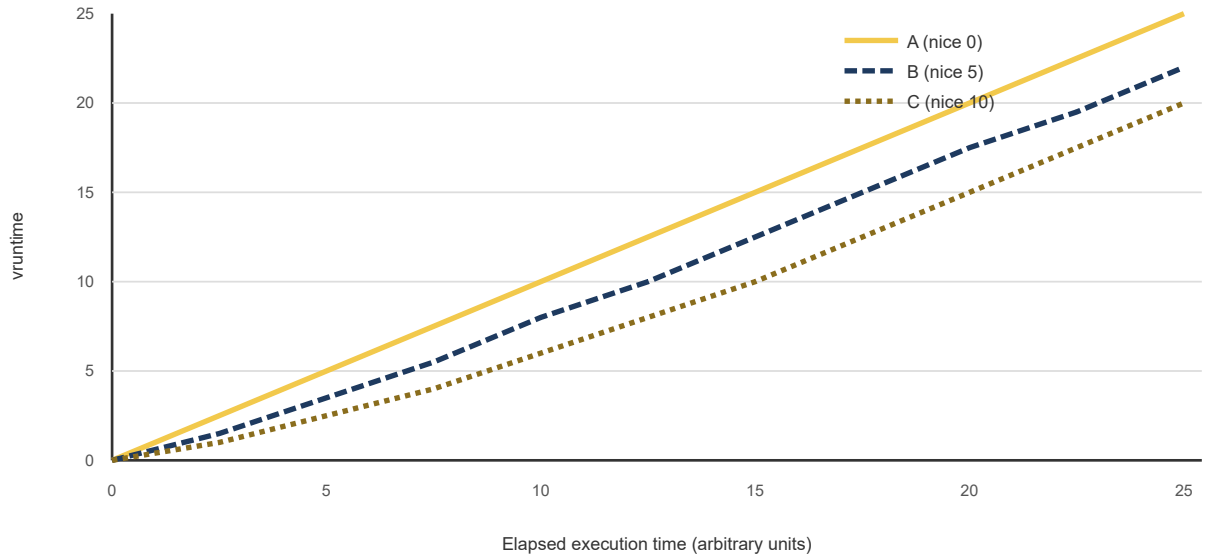


Figure 8.2 — Illustrative evolution of virtual runtime for three tasks with different weights. Higher-weight tasks accumulate virtual runtime more slowly. The slopes are pedagogical and do not represent a specific kernel trace.

8.3 Gantt Chart of Execution Intervals

Figure 8.3 provides a compact Gantt chart for the worked example, emphasizing that the scheduler alternates tasks to keep their virtual runtimes balanced.

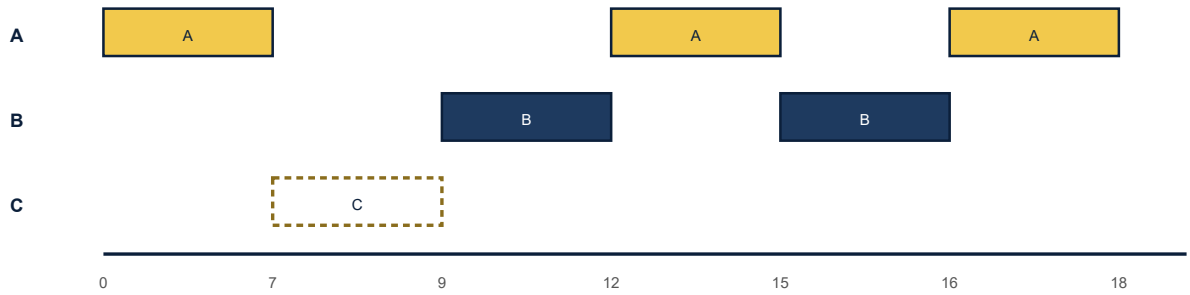


Figure 8.3 — Gantt chart of illustrative CFS execution intervals. The chart is schematic; real CFS slices are shorter and depend on the number of runnable entities and kernel tunables.

8.4 Qualitative Comparison of CFS, RM, and EDF

Figure 8.4 compares the three algorithms on qualitative dimensions. The scores are illustrative and are intended to support the discussion in Chapter 9, not to provide a quantitative benchmark.

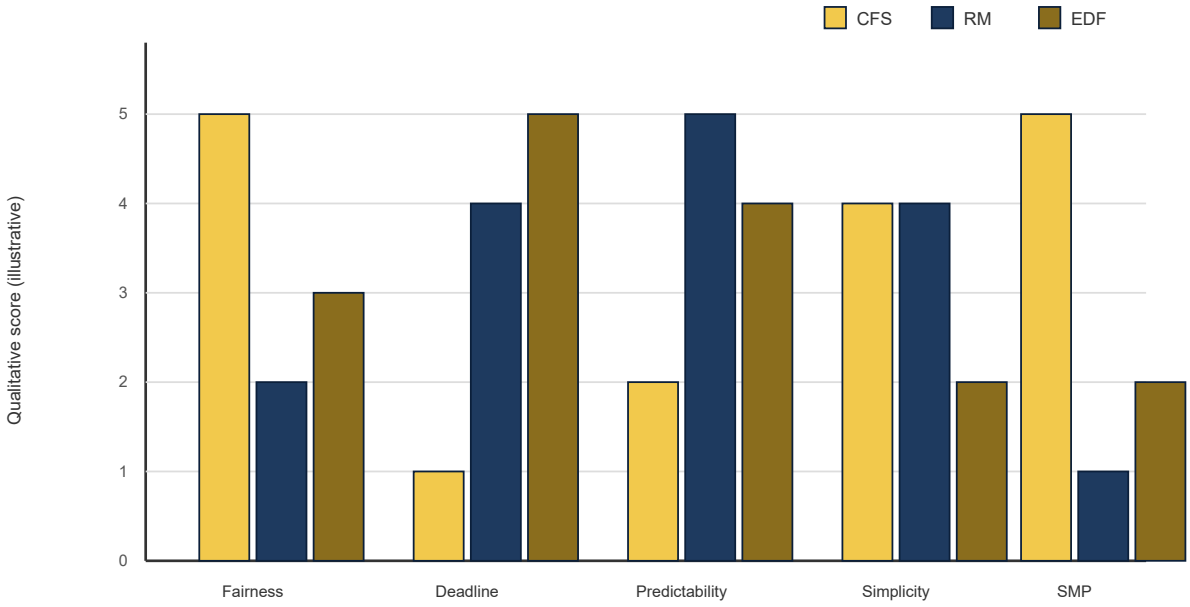


Figure 8.4 — Qualitative comparison of CFS, RM, and EDF on fairness, deadline support, predictability, simplicity, and SMP suitability. The scores are illustrative and should not be treated as measurements. CFS is strong on fairness and SMP scalability; RM and EDF are stronger on deadline support and predictability under their classical assumptions.

8.5 Load Balancing Across Four Cores

Figure 8.5 illustrates a load-balancing scenario. Before balancing, CPU 0 has three tasks and CPU 3 is idle; after balancing, one task has migrated, reducing the imbalance. The diagram is conceptual and omits the scheduling-domain hierarchy.

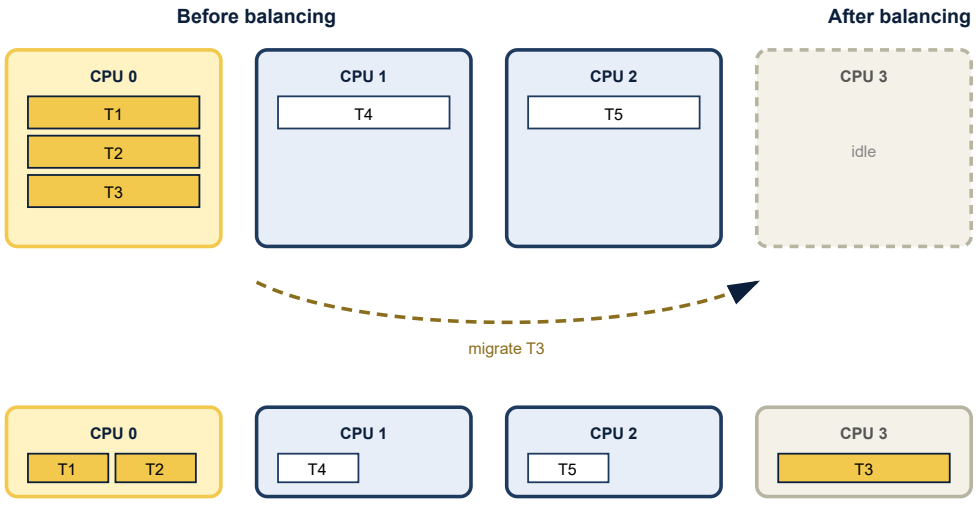


Figure 8.5 — Conceptual load balancing across four CPU cores. The balancer may migrate a task from a heavily loaded CPU to an idle CPU. Real balancing considers scheduling domains, cache affinity, and NUMA topology and may choose a different task or no migration at all.

DATA PROVENANCE WARNING

The charts in this chapter are **illustrative**. They were constructed from simplified models and rounded weights, not from kernel trace data or benchmark suites. No claim is made about statistical significance, confidence intervals, or reproducibility. For empirical performance data, readers should consult the original research literature, kernel mailing-list benchmarks, and workload-specific studies.

9. Comparison of CFS, Rate Monotonic (RM), and Earliest Deadline First (EDF)

9.1 CFS: Fair General-Purpose Scheduling

CFS is designed for general-purpose workloads where the objective is to give each runnable task a fair share of CPU time proportional to its weight. It does not require tasks to declare deadlines, periods, or worst-case execution times. It handles arbitrary task arrival patterns and blocking behavior. Its strengths are fairness, simplicity of configuration, and good behavior under mixed workloads. Its weakness is that it does not provide hard deadline guarantees: a fair share of CPU time is not the same as a guarantee that a task will complete by a specific time.

9.2 RM: Fixed-Priority Real-Time Scheduling

Rate Monotonic scheduling is a fixed-priority algorithm for periodic real-time tasks. Each task is assigned a priority inversely proportional to its period: shorter-period tasks receive higher priority. RM is optimal among fixed-priority algorithms for periodic tasks with deadlines equal to periods, under the classical assumptions of independent, preemptive tasks on a uniprocessor. The classical Liu and Layland utilization bound is:

$$U = \Sigma_{i=1}^{n} \frac{C_i}{T_i} \leq n(2^{1/n} - 1) \tag{9.1}$$

where C_i is the worst-case execution time of task i, T_i is its period, and n is the number of tasks. As n grows, the bound converges to $\ln 2 \approx 0.693$. This is a *sufficient* schedulability condition: if the total utilization is at or below the bound, the task set is guaranteed schedulable under RM. It is not necessary: some task sets with utilization above the bound are still schedulable, and exact response-time analysis can prove this. The bound assumes implicit deadlines, independent tasks, preemptive execution, and zero context-switch overhead[reference:46][reference:47].

9.3 EDF: Dynamic-Priority Real-Time Scheduling

Earliest Deadline First assigns priorities dynamically based on absolute deadlines: the task with the earliest deadline runs first. On a preemptive uniprocessor with independent periodic tasks and deadlines equal to periods, EDF is optimal: a task set is schedulable if and only if its total utilization is at most 1.0. This is a stronger result than the RM bound, but it depends on the same idealized assumptions. EDF can fail catastrophically under overload, and its behavior on multiprocessors is more complex. Practical systems add admission control, deadline enforcement, and throttling to prevent overload[reference:48][reference:49].

9.4 Detailed Comparison Table

Table 9.1 compares CFS, RM, and EDF across the criteria required for this study. The comparison highlights that CFS and the real-time algorithms occupy different design spaces: CFS optimizes fairness and generality, while RM and EDF optimize deadline satisfaction under explicit real-time assumptions.

Table 9.1 — Detailed comparison of CFS, RM, and EDF.

CRITERION	CFS	RATE MONOTONIC (RM)	EARLIEST DEADLINE FIRST (EDF)
Primary objective	Proportional fairness among runnable tasks	Meet deadlines for periodic tasks via fixed priorities	Meet deadlines via dynamic earliest-deadline priority
Scheduling model	General-purpose, event-driven, preemptive	Periodic task model, preemptive, fixed priority	Periodic or sporadic task model, preemptive, dynamic priority
Priority assignment	Weight from nice value; group weights	Shorter period → higher priority	Earlier absolute deadline → higher priority
Preemption	Preemptive; wake-up granularity reduces overhead	Preemptive	Preemptive
Deadline handling	No deadline concept	Implicit or explicit deadlines; analysis based on periods	Explicit deadlines; earliest deadline first
Fairness	Strong: virtual-runtime-based proportional sharing	Not a primary goal; fixed priorities can starve low-priority tasks	Not a primary goal; deadline-driven
Predictability	Statistical, workload-dependent; no hard bounds	Worst-case response-time analysis possible	Worst-case analysis possible under assumptions
Typical workload	Desktop, server, containers, general-purpose	Periodic control, embedded real-time	Periodic/sporadic real-time, adaptive systems
Scheduling overhead	O(log n) tree operations; per-CPU locks	Low; fixed-priority queue	Moderate; dynamic priority queue
Response-time focus	Average and tail latency under contention	Worst-case response time	Worst-case response time
Multiprocessor implications	Per-CPU queues, load balancing, affinity, NUMA	Global or partitioned; utilization bound changes	Global EDF, partitioned EDF, and hybrid approaches
Admission control	None inherent; cgroup bandwidth limits	Utilization-based test	Utilization ≤ 1 on uniprocessor; more complex on multiprocessors
Theoretical schedulability	Fairness bounds, not deadline guarantees	Sufficient bound n(2^(1/n) − 1); exact test possible	Utilization ≤ 1 sufficient and necessary on idealized uniprocessor
Strengths	Fair, general, scalable, easy to configure	Simple, predictable, optimal among fixed-priority	Optimal on idealized uniprocessor, higher utilization than RM
Weaknesses	No hard deadlines; latency depends on load	Low utilization bound; poor average-case fairness	Fragile under overload; multiprocessor complexity
Typical OS use	Linux fair class (CFS/EEVDF)	RTOS, embedded control, some Linux RT policies	Linux SCHED_DEADLINE, RTOS, research systems

9.5 Schedulability Analysis: Scope and Limitations

The RM bound and the EDF utilization bound are powerful theoretical results, but they rest on assumptions that are rarely fully satisfied in practice:

- **Periodic independent tasks.** Real tasks may have dependencies, shared resources, and sporadic arrivals.
- **Known worst-case execution times.** WCET analysis is difficult for modern processors with caches, pipelines, and SMT.
- **Preemptive execution with negligible overhead.** Context switches and cache effects introduce overhead that the idealized model ignores.
- **Uniprocessor scheduling.** Multiprocessor real-time scheduling is substantially more complex; global EDF and partitioned approaches have different bounds and anomalies.
- **No overload.** EDF's optimality assumes the system is not overloaded; under overload, deadlines can be missed in a way that is difficult to predict.

CFS does not attempt to satisfy these assumptions and therefore does not provide the corresponding guarantees. Conversely, RM and EDF are not designed to provide proportional fairness among arbitrary tasks. The choice between them depends on the application's requirements: if deadlines are hard and the task model is known, RM or EDF with appropriate analysis may be appropriate; if the workload is general-purpose and fairness is the primary goal, CFS (or EEVDF) is a better fit.

9.6 When Each Approach Is Appropriate

CFS is appropriate for desktops, servers, containers, and mixed workloads where no task has a hard deadline and the goal is to share CPU capacity fairly and responsively. **RM** is appropriate for periodic real-time systems with fixed priorities, especially when worst-case response-time analysis is required and the utilization is within the RM bound. **EDF** is appropriate for real-time systems that can benefit from dynamic priorities and higher utilization, provided that overload is prevented and the task model is understood. In Linux, `SCHED_DEADLINE` provides an EDF-based policy with admission control, while `SCHED_FIFO` and `SCHED_RR` provide fixed-priority real-time policies[reference:50].

10. Limitations and Disadvantages of CFS

10.1 Lack of Inherent Hard Real-Time Deadline Guarantees

CFS is a fairness-oriented scheduler. It does not accept deadline parameters, does not perform admission control based on deadlines, and does not guarantee that a task will complete by a specified time. Under heavy contention, a fair-class task may experience scheduling latency that is unacceptable for a hard real-time system. This is not a bug; it is a consequence of the design goals. Real-time guarantees in Linux are provided by the real-time and deadline scheduling classes, not by CFS.

10.2 Fairness versus Deadline Satisfaction

Fairness and deadline satisfaction are different objectives. A scheduler can be perfectly fair in the long run while still missing every deadline if the deadlines are shorter than the fair-share interval. Conversely, a deadline-driven scheduler can meet all deadlines while being highly unfair to tasks without deadlines. CFS's virtual-runtime mechanism ensures that no runnable task is starved of CPU time, but it does not ensure that any task completes by a particular time. This distinction is fundamental and should be emphasized in any comparison of CFS with RM or EDF.

10.3 Scheduling Latency and Latency-Sensitive Workloads

CFS's scheduling latency depends on the number of runnable tasks, the target latency, and the minimum granularity. When many tasks are runnable, the interval between successive executions of a given task can grow, which may be unacceptable for audio, video, or control loops. The wake-up granularity reduces preemption overhead but can also delay a newly woken latency-sensitive task. EEVDF addresses some of these issues by allowing tasks to request shorter slices and by using virtual deadlines, but even EEVDF is not a hard real-time scheduler[reference:51].

10.4 Run-Queue Synchronization and Contention

Per-CPU run queues improve scalability but require synchronization for migration, load balancing, and parameter changes. Under workloads with many short-lived tasks or frequent migrations, lock contention can become a bottleneck. The kernel mitigates this with fine-grained locking and RCU, but contention cannot be eliminated entirely. This is an implementation-level limitation rather than a conceptual one, but it affects real-world performance.

10.5 Migration Overhead and Cache Disruption

Load balancing may migrate tasks between CPUs, which can improve utilization but disrupt cache locality. A migrated task may need to reload its working set, causing cache misses and TLB misses. CFS attempts to balance migration benefits against these costs, but the trade-off is workload-dependent. In some cases, keeping a task on a busy CPU with a warm cache is better than migrating it to an idle CPU with a cold cache.

10.6 NUMA Effects

On NUMA systems, migrating a task across nodes can increase memory latency if the task's data remains on the original node. CFS's NUMA-aware load balancing considers locality, but perfect NUMA optimization is difficult because it depends on memory access patterns that the scheduler cannot fully observe. Workloads with large working sets and irregular access patterns may experience suboptimal performance.

10.7 Fairness Accounting Complexities

CFS's fairness accounting is more complex than the simplified Equation 3.2 suggests. Group scheduling, load tracking, wake-up placement, and migration adjustments all affect the effective virtual runtime. Bugs or misconfigurations in these mechanisms can produce fairness deviations. The complexity also makes it difficult for administrators to predict exact CPU shares in hierarchical configurations.

10.8 Group Scheduling and Hierarchical Allocation Complexities

Group scheduling introduces hierarchical weights, bandwidth limits, and throttling. These mechanisms are powerful but complex. A misconfigured cgroup can receive less CPU than intended, or can be throttled even when the system is not fully loaded. The interaction between group weights and per-task weights is not always intuitive, and debugging requires familiarity with cgroup v2 and CFS bandwidth control[reference:52][reference:53].

10.9 Workload-Dependent Behavior

CFS's performance is workload-dependent. A configuration that works well for a web server may be suboptimal for a build farm or a real-time control system. There is no single "best" set of tunables for all workloads. Administrators must understand their workload and monitor scheduling behavior to achieve good results.

10.10 Kernel-Version Differences

The fair scheduling class has changed over time. Classic CFS behavior in Linux 5.x differs from EEVDF behavior in Linux 6.6 and later. Sysctls and tunables have been added, removed, or renamed. Documentation and advice written for one kernel version may be inaccurate for another. This is a significant practical limitation: scheduling behavior cannot be understood without specifying the kernel version and configuration[reference:54][reference:55].

10.11 Limitations of Simplified Theoretical Models

Simplified models such as Equation 3.2 and the RM/EDF bounds are useful for understanding, but they omit many real-world effects: cache behavior, SMT contention, memory bandwidth, I/O interference, and virtualization overhead. Conclusions drawn from idealized models should be validated empirically before being applied to production systems. This paper's illustrative examples are explicitly pedagogical and should not be treated as performance predictions.

11. Starvation, Deadlock, and Priority Inversion

11.1 Starvation

Definition. Starvation occurs when a task is runnable but is indefinitely prevented from making progress because other tasks are always selected first. In a pure fair scheduler, starvation among fair-class tasks is unlikely under normal conditions because the scheduler always selects the task with the smallest virtual runtime. A task that has been waiting accumulates a relatively small virtual runtime, so it should eventually become the leftmost node and be selected. However, starvation can still occur for reasons outside the fair scheduler's control:

- **Higher-priority scheduling classes.** A continuous stream of real-time tasks can starve fair-class tasks indefinitely unless real-time throttling is configured[reference:56].
- **CPU bandwidth throttling.** A cgroup that exceeds its `cpu.max` quota may be throttled, preventing its tasks from running even if they are runnable[reference:57].
- **Resource deadlock.** A task blocked on a lock or I/O operation cannot make progress regardless of how fairly CPU time is distributed.
- **Priority inversion.** A high-priority task may be indirectly delayed by a lower-priority task holding a needed resource.

CFS-style fairness reduces the risk of persistent CPU starvation among runnable fair-class tasks, but it does not make starvation impossible. The distinction between starvation, delayed execution, throttling, blocking, and starvation caused by other mechanisms is important for correct diagnosis.

11.2 Deadlock

Definition. Deadlock is a state in which a set of tasks are each waiting for a resource held by another task in the set, so none can proceed. Deadlock is primarily a resource-allocation and synchronization problem, not a consequence of the CPU-selection policy. The four Coffman conditions are necessary for deadlock:

1. **Mutual exclusion.** At least one resource is held in a non-shareable mode.
2. **Hold and wait.** A task holds at least one resource and is waiting for additional resources held by other tasks.
3. **No preemption.** Resources cannot be forcibly taken from a task; they must be released voluntarily.
4. **Circular wait.** There exists a circular chain of tasks, each waiting for a resource held by the next.

A task blocked by a deadlock remains unable to make progress regardless of how fairly the CPU is scheduled. CFS can ensure that the task receives CPU time when it becomes runnable, but it cannot resolve the resource cycle. Deadlock prevention, avoidance, detection, and recovery are separate concerns that belong to synchronization and resource-management subsystems.

11.3 Priority Inversion

Definition. Priority inversion occurs when a high-priority task is indirectly delayed by a lower-priority task that holds a resource needed by the high-priority task, while a medium-priority task preempts the lower-priority task. The classical example involves three tasks: *H* (high priority), *M* (medium priority), and *L* (low priority). *L* acquires a lock. *H* becomes runnable and tries to acquire the same lock, but blocks because *L* holds it. *M* becomes runnable and, being higher priority than *L*, preempts *L*. Now *H* is waiting for *L*, but *L* cannot run because *M* is running. The high-priority task is effectively delayed by the medium-priority task.

Priority inversion differs from starvation and deadlock. It is not caused by the CPU scheduler ignoring a runnable task; it is caused by the interaction between priority-based scheduling and resource locking. Solutions include **priority inheritance** (the lock holder temporarily inherits the priority of the highest-priority waiter) and **priority ceiling** protocols. These mechanisms are particularly important in real-time systems and are supported by RT-mutexes in Linux. CFS itself does not implement priority inheritance for fair-class tasks, so priority inversion involving fair-class tasks must be addressed through synchronization design.

11.4 Comparison Table

Table 11.1 — Starvation versus deadlock versus priority inversion.

ASPECT	STARVATION	DEADLOCK	PRIORITY INVERSION
Core problem	A task never gets CPU or resources	Circular wait; no task can proceed	High-priority task delayed by lower-priority lock holder
Cause	Unfair selection, higher-priority classes, throttling, blocking	Resource cycle satisfying Coffman conditions	Priority scheduling + non-preemptible resource held by low-priority task
Relationship to CFS	Fair accounting reduces risk among fair-class runnable tasks, but does not eliminate all causes	Not caused by CPU selection; a synchronization problem	Not caused by CFS alone; a synchronization and priority interaction problem
Typical symptom	Task waits indefinitely; low CPU time	System or task set hangs; no progress	High-priority task misses deadline despite being runnable
Mitigation	Fair scheduling, real-time throttling, cgroup limits, fixing resource bottlenecks	Deadlock prevention, avoidance, detection, recovery	Priority inheritance, priority ceiling, careful lock design

12. Security, Reliability, and Practical Considerations

12.1 Scheduler Isolation Boundaries

The scheduler is part of the kernel's trusted computing base. A bug in scheduling code can cause crashes, data corruption, or denial of service. CFS and its supporting infrastructure are therefore subject to strict kernel coding and review standards. User-space interfaces such as nice, affinity, and cgroup controls are mediated by permission checks. Unprivileged users can change nice values within limits and set affinity for their own tasks, but cannot arbitrarily change scheduling policies or other users' tasks without appropriate privileges.

12.2 Resource Exhaustion and Denial-of-Service Considerations

A malicious or buggy task that creates many runnable threads can impose scheduling overhead and degrade system responsiveness. CFS's per-CPU run queues and O(log n) operations limit the cost of individual scheduling decisions, but the aggregate cost of many tasks can still be significant. Cgroup CPU quotas and bandwidth limits can contain the impact of a runaway group. Real-time throttling prevents real-time tasks from consuming 100% of CPU time indefinitely[reference:58]. These mechanisms are not absolute security guarantees; they are resource-management tools that must be configured correctly.

12.3 CPU Quotas and cgroup-Based Resource Controls

The cgroup v2 CPU controller provides two principal mechanisms: **weight** (proportional sharing) and **bandwidth** (absolute limit). Weight is set through `cpu.weight` and influences the group's share when the CPU is contended. Bandwidth is set through `cpu.max`, which specifies a quota and period; a group that exceeds its quota is throttled until the next period[reference:59]. These controls are essential in container environments, where multiple tenants share a host. Misconfiguration can lead to throttling even when the host is not fully utilized, so monitoring and tuning are important.

12.4 Scheduling Policy Configuration

Linux exposes scheduling policy through `sched_setscheduler(2)`, `sched_setattr(2)`, and command-line tools. The fair policies are `SCHED_NORMAL`, `SCHED_BATCH`, and `SCHED_IDLE`. Real-time policies are `SCHED_FIFO` and `SCHED_RR`, and deadline scheduling uses `SCHED_DEADLINE`. Changing policy requires appropriate privileges (typically `CAP_SYS_NICE`). Administrators should be cautious: assigning `SCHED_FIFO` to a busy task can starve fair-class tasks and make the system unresponsive.

12.5 Kernel Tuning Trade-offs

CFS exposes several tunables, including `sched_latency_ns`, `sched_min_granularity_ns`, and `sched_wakeup_granularity_ns` [reference:60]. Lowering latency improves responsiveness but increases context-switching overhead. Raising it improves throughput for CPU-bound workloads but

can harm interactive performance. The optimal values depend on the workload and hardware. Kernel versions may change defaults or remove tunables, especially with the transition to EEVDF, so tuning advice should be version-specific.

12.6 Monitoring CPU Utilization and Latency

Tools such as `top`, `htop`, `pidstat`, and `perf sched` can reveal CPU utilization, run-queue length, context-switch rates, and scheduling latency. `/proc//sched` exposes per-task scheduling statistics. Monitoring is essential for detecting starvation, excessive migrations, and throttling. However, interpreting these tools requires an understanding of the scheduler version and configuration; a metric that indicates a problem on one kernel may be normal on another.

12.7 Reliability in Desktop, Server, and Container Environments

CFS has been widely deployed on desktops, servers, and embedded systems. Its fairness and scalability make it suitable for mixed workloads, and its integration with cgroups makes it usable in container orchestrators. Reliability concerns differ by environment: desktops prioritize responsiveness, servers prioritize throughput and tail latency, and containers prioritize isolation and predictable allocation. No single configuration is optimal for all three. EEVDF's introduction reflects the ongoing effort to improve latency without sacrificing fairness, particularly for latency-sensitive tasks in general-purpose environments[reference:61].

13. Practical Linux Demonstration

This chapter provides a reproducible educational demonstration. The commands are non-destructive and can be run on a typical Linux system. Some operations require root privileges or `CAP_SYS_NICE`; these are noted. The goal is to observe scheduling behavior, not to benchmark the kernel.

13.1 Inspecting Processes with `ps`

```
ps -eo pid,ni,pri,psr,pcpu,comm --sort=-pcpu | head -20
```

The `ni` column shows the nice value, `pri` shows the kernel priority, `psr` shows the CPU on which the process last ran, and `pcpu` shows CPU utilization. This command gives a snapshot of which tasks are consuming CPU and at what nice level. It does not show scheduling latency or run-queue length.

13.2 Observing Run-Queue Behavior with `top` or `htop`

```
top -o %CPU
# or
htop
```

`top` displays load averages, CPU utilization, and per-process CPU usage. The load average includes runnable and uninterruptible tasks, so it is not a pure run-queue length metric. `htop` provides a more interactive view and can show per-CPU utilization. These tools help identify whether the system is CPU-saturated, but they do not directly expose `vruntime` or scheduling latency.

13.3 Changing Nice Values with `nice` and `renice`

```
# Start a CPU-bound process at nice 10 (lower priority)
nice -n 10 sha256sum /dev/zero

# Change the nice value of an existing process (may require privileges)
renice -n 5 -p <PID>

# Inspect the nice value
ps -o pid,ni,comm -p <PID>
```

Changing the nice value changes the task's weight and therefore its proportional CPU share. It does *not* change the scheduling class. A nice-19 task is still a fair-class task and can be preempted by real-time tasks. Negative nice values require privileges and can starve other tasks if misused.

13.4 Setting CPU Affinity with `taskset`

```
# Run a command pinned to CPU 0
taskset -c 0 sha256sum /dev/zero

# Change affinity of an existing process (may require privileges)
taskset -pc 1 <PID>

# Inspect affinity
taskset -pc <PID>
```

`taskset` manipulates the CPU affinity mask. Pinning a task to one CPU can improve cache locality but may leave other CPUs idle. Affinity is a hard constraint respected by CFS's load balancer, so it should be used with care[reference:62].

13.5 Reading `/proc` Scheduling Information

```
# Per-task scheduling statistics
cat /proc/<PID>/sched

# Global scheduler statistics
cat /proc/schedstat

# Cgroup CPU controller settings (cgroup v2)
cat /sys/fs/cgroup/<group>/cpu.weight
cat /sys/fs/cgroup/<group>/cpu.max
```

`/proc//sched` provides fields such as `se.sum_exec_runtime`, `se.vruntime`, and `nr_migrations`, which can help diagnose scheduling behavior. The exact fields depend on the kernel version. `/proc/schedstat` provides per-CPU scheduling statistics.

13.6 Real-Time Policies with `chrt`

```
# Show the scheduling policy and priority of a process
chrt -p <PID>

# Run a command with SCHED_FIFO priority 50 (requires privileges)
sudo chrt -f 50 <command>

# Run a command with SCHED_RR priority 20 (requires privileges)
sudo chrt -r 20 <command>
```

`chrt` changes the scheduling policy and real-time priority. `SCHED_FIFO` tasks run until they block or yield; `SCHED_RR` tasks use a time slice. These policies can starve fair-class tasks, so they should be used only when necessary and with real-time throttling configured[reference:63]. Changing a task's

nice value is fundamentally different from changing its scheduling class: nice affects fair-class weight, while `chrt` can move the task into the real-time class.

13.7 Interpreting Output and Safety

When interpreting scheduling output, remember that the kernel version matters. A system running Linux 6.6 or later uses EEVDF in the fair class, so the selection behavior and some tunables differ from classic CFS. Do not run CPU-bound test programs for long periods on production systems, and do not assign real-time priorities to untrusted processes. The commands above are intended for short, controlled experiments on a test machine or a disposable virtual machine.

Table 13.1 — Linux scheduling tools and their purposes.

TOOL	PURPOSE	PRIVILEGES / NOTES
`ps`	Snapshot of processes, nice values, CPU usage	Unprivileged
`top` / `htop`	Interactive CPU and process monitoring	Unprivileged
`nice`	Start a process with a specified nice value	Negative values require privileges
`renice`	Change the nice value of a running process	May require privileges
`taskset`	Get or set CPU affinity	Changing affinity of other users' tasks may require privileges
`/proc/<pid>/sched`	Per-task scheduling statistics	Readable by owner or root
`/proc/schedstat`	Global scheduling statistics	Readable by root or authorized users
`chrt`	Get or set scheduling policy and real-time priority	Real-time policies require privileges
`cpu.weight` / `cpu.max`	cgroup v2 CPU weight and bandwidth control	Requires cgroup delegation or root

14. Discussion and Future Directions

14.1 The Evolution of Linux Scheduling

Linux scheduling has moved from simple priority arrays to fair-share scheduling and, more recently, toward deadline-aware fair scheduling. The O(1) scheduler prioritized constant-time selection but relied on heuristics to infer interactivity. CFS replaced those heuristics with a virtual-clock model that made fairness explicit and scalable. EEVDF retains the proportional-share foundation of CFS but adds lag and virtual deadlines to improve latency control and reduce reliance on tunables[reference:64]. This evolution reflects a broader trend: general-purpose schedulers are incorporating ideas from real-time scheduling to improve responsiveness without abandoning fairness.

14.2 The Transition from CFS to EEVDF

The transition to EEVDF is significant because it changes the fair class's selection algorithm and removes several CFS heuristics. EEVDF allows tasks to request specific time slices through `sched_setattr(2)`, and it uses virtual deadlines to prioritize latency-sensitive tasks. It also uses a decaying lag mechanism to prevent sleeping tasks from exploiting the system[reference:65]. The transition began in Linux 6.6 and continued in later kernels. Administrators and developers should verify which algorithm their kernel uses before applying tuning advice written for classic CFS.

14.3 Why Kernel Version Matters

Scheduling behavior is not a property of "Linux" in the abstract; it is a property of a specific kernel version and configuration. The fair class changed in 6.6, cgroup interfaces evolved, and tunables were added or removed. A paper that describes CFS without specifying a kernel version risks conflating classic CFS, EEVDF, and intermediate states. This paper has attempted to be explicit about version-dependent claims, but readers should consult the documentation for their exact kernel.

14.4 Balancing Fairness, Latency, Throughput, and Scalability

The central challenge of general-purpose scheduling is balancing competing objectives. Fairness requires giving every runnable task a turn; latency requires giving some tasks priority; throughput favors long runs and cache locality; scalability requires per-CPU structures and minimal cross-CPU synchronization. CFS made a particular trade-off: it prioritized fairness and scalability, accepted heuristic latency control, and relied on scheduling classes for real-time guarantees. EEVDF adjusts the trade-off by introducing explicit latency parameters and deadline-based selection. Neither approach is universally optimal; the best choice depends on the workload and the system's requirements.

14.5 Scheduling in Virtualized and Containerized Environments

Virtualization and containers add new dimensions to scheduling. A virtual machine's guest scheduler operates on virtual CPUs that are themselves scheduled by the host. A container's tasks are scheduled by the host kernel but constrained by cgroup weights and quotas. These layers can interact in

complex ways: a guest may believe it has four CPUs while the host schedules its virtual CPUs on two physical cores; a container may be throttled by `cpu.max` even though the host is not saturated. Group scheduling and bandwidth control are essential tools in these environments, but they require careful configuration and monitoring[reference:66].

14.6 Potential Directions for Future Scheduling Research

Several directions are active areas of research and development. First, latency-aware fair scheduling—exemplified by EEVDF—is likely to continue evolving, with better mechanisms for specifying and enforcing latency requirements without compromising fairness. Second, heterogeneous CPU architectures (big.LITTLE, performance/efficiency cores) require schedulers that understand capacity differences and energy trade-offs. Third, memory-side effects—NUMA locality, cache partitioning, memory bandwidth—are increasingly important as core counts grow. Fourth, scheduling for accelerators and heterogeneous compute is an open problem. Fifth, formal verification and testing of scheduler invariants could improve reliability. These directions are speculative and should be distinguished from established implementation facts.

15. Conclusion

The Completely Fair Scheduler is one of the most influential general-purpose CPU schedulers ever deployed. Its central idea—model an ideal multi-tasking CPU on real hardware and track each task's virtual runtime relative to its weight—is simple, elegant, and effective. By selecting the runnable entity with the smallest virtual runtime, classic CFS provides proportional fairness without requiring tasks to be classified as interactive or CPU-bound. The red-black tree gives efficient O(log n) insertion and deletion, while per-CPU run queues and scheduling domains support scalable multiprocessing.

CFS is not, however, a real-time scheduler. It does not accept deadlines, does not perform deadline-based admission control, and does not guarantee bounded worst-case latency. Its fairness is a long-run property that can deviate in short intervals, and its behavior depends on kernel version, configuration, and workload. The comparison with Rate Monotonic and Earliest Deadline First in Chapter 9 showed that CFS, RM, and EDF occupy different design spaces: CFS optimizes fairness and generality, RM provides fixed-priority real-time guarantees under classical assumptions, and EDF provides dynamic-priority optimality on idealized uniprocessors. None is universally superior; the appropriate choice depends on the application's requirements.

The transition from classic CFS to EEVDF in Linux 6.6 marks another step in the evolution of Linux scheduling. EEVDF retains proportional fairness but adds lag and virtual deadlines to improve latency control and reduce heuristic tuning. This transition reinforces the importance of kernel-version awareness: claims about "the Linux scheduler" must be qualified by the kernel version and configuration.

In summary, CFS demonstrated that fairness can be a first-class scheduling objective in a general-purpose operating system, and it provided a foundation for the deadline-aware fair scheduling that followed. Its strengths—proportional sharing, scalability, and integration with Linux's scheduling-class hierarchy—are substantial. Its limitations—lack of hard real-time guarantees, workload-dependent latency, and implementation complexity—are equally important to understand. A balanced assessment recognizes CFS as a major engineering achievement whose ideas continue to shape Linux scheduling, while acknowledging that no single algorithm can satisfy all scheduling objectives simultaneously.

Table 15.1 — Summary of the main findings.

FINDING	SUMMARY
CFS model	Approximates an ideal multi-tasking CPU using weighted virtual runtime and a red-black tree.
Selection rule	Classic CFS selects the entity with the smallest `vruntime`; EEVDF selects the eligible entity with the earliest virtual deadline.
Fairness	Long-run CPU shares are proportional to weights; short-run deviations are expected.
Multicore	Per-CPU run queues, load balancing, affinity, and NUMA-aware scheduling domains.
Real-time	CFS does not provide hard deadline guarantees; real-time and deadline classes exist for that purpose.
RM vs EDF	RM is fixed-priority with a sufficient utilization bound; EDF is dynamic-priority and optimal on idealized uniprocessors.
Version awareness	Linux 6.6 replaced classic CFS selection with EEVDF in the fair class; tuning and behavior differ by version.
Limitations	No hard deadlines, workload-dependent latency, migration/cache overhead, and configuration complexity.
Practical use	CFS/EEVDF are suitable for desktops, servers, and containers; real-time workloads require appropriate scheduling classes and analysis.

References

1. Linux Kernel Documentation, "CFS Scheduler," *Documentation/scheduler/sched-design-CFS.rst*. [Online]. Available: https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html
2. Linux Kernel Documentation, "EEVDF Scheduler," *Documentation/scheduler/sched-eevdf.rst*. [Online]. Available: https://www.kernel.org/doc/html/latest/scheduler/sched-eevdf.html
3. KernelNewbies, "Linux 6.6," 29 October 2023. [Online]. Available: https://kernelnewbies.org/Linux_6.6
4. C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973, doi: 10.1145/321738.321743.
5. I. Stoica and H. Abdel-Wahab, "Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation," Technical Report TR-95-22, Department of Computer Science, Old Dominion University, 1995.
6. Linux man-pages project, *sched(7) — overview of CPU scheduling*. [Online]. Available: https://man7.org/linux/man-pages/man7/sched.7.html
7. Linux Kernel Documentation, "Control Group v2," *Documentation/admin-guide/cgroup-v2.rst*. [Online]. Available: https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html
8. Linux Kernel Documentation, "CFS Bandwidth Control," *Documentation/scheduler/sched-bwc.rst*. [Online]. Available: https://www.kernel.org/doc/html/latest/scheduler/sched-bwc.html
9. Linux man-pages project, *nice(1)*, *renice(1)*, *taskset(1)*, *chrt(1)*. [Online]. Available: https://man7.org/linux/man-pages/
10. J. Corbet, "An EEVDF CPU scheduler for Linux," *LWN.net*, 2023. [Online]. Available: https://lwn.net/Articles/925371/
11. Linux kernel source tree, `kernel/sched/fair.c` and `kernel/sched/core.c`. [Online]. Available: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/
12. R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.
13. D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2005.
14. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
15. P. Zijlstra, "EEVDF scheduler," Linux Kernel Mailing List, 2023. [Online]. Available: https://lore.kernel.org/lkml/
16. Linux Kernel Documentation, "Scheduler Statistics," *Documentation/scheduler/sched-stats.rst*. [Online]. Available: https://www.kernel.org/doc/html/latest/scheduler/
17. Linux Kernel Documentation, "Per-Entity Load Tracking," *Documentation/scheduler/sched-pelt.c* and related kernel documentation. [Online]. Available: https://www.kernel.org/doc/html/latest/scheduler/
18. M. Tim Jones, "Inside the Linux 2.6 Completely Fair Scheduler," IBM developerWorks, 2009. [Online]. Available: https://developer.ibm.com/articles/l-cfs/
19. Linux Kernel Documentation, "Real-Time Group Scheduling," *Documentation/scheduler/sched-rt-group.rst*. [Online]. Available: https://www.kernel.org/doc/html/latest/scheduler/sched-rt-group.html
20. Linux Kernel Documentation, "Deadline Task Scheduling," *Documentation/scheduler/sched-deadline.rst*. [Online]. Available: https://www.kernel.org/doc/html/latest/scheduler/sched-deadline.html